



TIỂU LUẬN CUỐI KỲ

MÔN HỌC: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

HỆ THỐNG HỎI ĐÁP FAQ DÀNH CHO SINH VIÊN HCM-UTE

Giảng viên: ThS. Đoàn Minh Trí

Danh sách sinh viên thực hiện

Mã số SV	Họ và tên	Mức độ đóng góp (%)
23110280	Huỳnh Thanh Nhân	100
23110300	Trương Tấn Sang	100
23110327	Huỳnh Ngọc Thắng	100

TP. Hồ Chí Minh, tháng 03 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm xin được gửi lời cảm ơn chân thành nhất đến ThS. Đoàn Minh Trí. Thầy đã tận tâm hướng dẫn nhóm chúng em từ lúc chọn đề tài cho đến lúc hoàn thành đồ án. Qua những bài học, lời khuyên và sự đóng góp của thầy đã giúp nhóm hiểu hơn về môn học Xử lý ngôn ngữ tự nhiên cũng như trong quá trình hoàn thành đồ án cuối kỳ.

Do thời gian đợt học lần này đã ngắn đi nên cùng với những kiến thức và bài học tiếp thu được, nhóm đã cố gắng thực hiện đề tài hết mình, cùng nhau thực hiện một cách tốt nhất với khả năng. Chính vì thế trong quá trình đôi khi sẽ có những sai sót không mong muốn, từ đó nhóm hi vọng sẽ nhận được sự góp ý từ thầy để nhóm có thể sửa chữa và rút ra bài học hướng đến mục tiêu là hoàn thiện các đồ án sau này.

Một lần nữa nhóm xin chân thành cảm ơn thầy. Kính chúc thầy thật nhiều sức khỏe, niềm vui trong sự nghiệp trồng người. Nhóm chúng em xin cảm ơn thầy.

TP. Hồ Chí Minh, ngày 29 tháng 03 năm 2026

Nhóm sinh viên thực hiện

Huỳnh Thanh Nhân

Trương Tấn Sang

Huỳnh Ngọc Thắng

MỤC LỤC

LỜI CẢM ƠN.....	
DANH MỤC BẢNG	
DANH MỤC HÌNH ẢNH.....	
PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu đề tài.....	1
3. Yêu cầu đặt ra.....	1
4. Phạm vi và đối tượng	1
5. Bố cục bài báo cáo	2
PHẦN NỘI DUNG.....	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	3
1.1. Tổng quan về Xử lý ngôn ngữ tự nhiên (Natural Language Processing)	3
1.1.1. Giới thiệu Xử lý ngôn ngữ tự nhiên	3
1.1.2. Tokenization và Word Segmentation tiếng Việt (PyVi).....	3
1.1.3. Text Embedding và Vectorization	4
1.2. Các thuật toán và mô hình liên quan.....	4
1.2.1. TF-IDF và Cosine Similarity	4
1.2.2. Mô hình ngôn ngữ PhoBERT	5
1.2.3. Mô hình ngôn ngữ lớn (LLM) và họ Qwen	6
1.3. Bài toán QA (Question Answering).....	7
1.3.1. Tổng quan hệ thống Hỏi đáp (QA)	7
1.3.2. Kiến trúc Retrieval-Augmented Generation (RAG)	7
1.3.3. Cơ sở dữ liệu Vector (ChromaDB)	8
1.4. Các thư viện sử dụng	8
1.4.1. Nhóm thư viện học sâu và xử lý mô hình ngôn ngữ.....	8
1.4.2. Nhóm thư viện xử lý dữ liệu và NLP tiếng Việt.....	9
1.4.3. Nhóm thư viện cơ sở hạ tầng và triển khai ứng dụng.....	10
CHƯƠNG 2: MÔ TẢ BÀI TOÁN VÀ HỆ THỐNG DỮ LIỆU	11
2.1. Phát biểu bài toán.....	11
2.1.1. Định nghĩa bài toán	11
2.1.2. Xác định Input và Output.....	11
2.2. Thu thập và phân tích dữ liệu	12

2.2.1. Nguồn dữ liệu.....	12
2.2.2. Đặc điểm ngôn ngữ, số lượng mẫu và lớp	12
2.2.3. Phân tích phân bố dữ liệu.....	13
2.3. Tiền xử lý dữ liệu.....	14
2.3.1. Sanitize Unicode và chuẩn hóa NFC	14
2.3.2. Làm sạch văn bản.....	15
2.3.3. Kỹ thuật sinh dữ liệu Paraphrase	16
CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT VÀ KIẾN TRÚC HỆ THỐNG	18
3.1. Tổng quan các phương pháp tiếp cận	18
3.1.1. Baseline 1: FAQ Retrieval truyền thống.....	18
3.1.2. Baseline 2: Generative QA (Dùng trực tiếp LLM)	19
3.1.3. Baseline 3: Thuần RAG (Retrieval-Augmented Generation)	19
3.2. Kiến trúc đề xuất: Hybrid FAQ + Generative RAG	20
3.2.1. Luồng xử lý giao tiếp (Routing Mechanism – Ngưỡng Threshold) ..	21
3.2.2. Pipeline truy xuất – sinh hoàn chỉnh.....	22
3.3. Huấn luyện và tinh chỉnh mô hình (Fine-tuning)	24
3.3.1. Fine-tune PhoBERT cho FAQ Retrieval bằng tập dữ liệu Paraphrase	24
3.3.2. Cấu hình Loss, Optimizer và hàm đánh giá.....	27
3.3.3. Fine-tune mô hình sinh (Qwen 2.5 7B).....	28
CHƯƠNG 4: THỰC NGHIỆM, KẾT QUẢ VÀ SO SÁNH.....	31
4.1. Môi trường và thiết lập thực nghiệm	31
4.2. Các tiêu chí đánh giá.....	32
4.2.1. Đánh giá truy xuất: Precision@k (P@1, P@3, P@5).....	32
4.2.2. Cosine Similarity và Margin Analysis	32
4.3. Kết quả đánh giá và so sánh phương pháp.....	33
4.3.1. So sánh TF-IDF, PhoBERT gốc và PhoBERT Fine-tuned.....	33
4.3.2. Phân tích kết quả fine-tune mô hình sinh.....	37
4.4. Thảo luận và Phân tích lỗi (Error Analysis)	39
4.4.1. Các trường hợp mô hình tốt / xấu (Lỗi thường gặp, OOV)	39
4.4.2. Hạn chế.....	41
CHƯƠNG 5: DEMO ỨNG DỤNG	42
5.1. Chi tiết triển khai	42

5.1.1. Backend API bằng Flask.....	42
5.1.2. Giao diện Web (UI/UX).....	43
5.2. Kịch bản sử dụng thực tế	44
PHẦN KẾT LUẬN	48
1. Đánh giá kết quả.....	48
1.1. Ưu điểm.....	48
1.2. Nhược điểm.....	49
2. Hướng phát triển	49
TÀI LIỆU THAM KHẢO	
BẢNG KHỐI LƯỢNG CÔNG VIỆC	

DANH MỤC BẢNG

Bảng 2.1: Xác định Input và Output của bài toán	11
Bảng 2.2: Thống kê số lượng mẫu dữ liệu	13
Bảng 2.3: Mẫu kết quả tiền xử lý dữ liệu	16
Bảng 4.1: Cấu hình môi trường và thiết lập thực nghiệm	31
Bảng 4.2: Kết quả so sánh TF-IDF, PhoBERT gốc và PhoBERT Fine-tuned.....	34
Bảng 4.3: Cấu hình fine-tune Qwen2.5-7B với LoRA.....	38
Bảng 4.4: Kết quả định lượng trên tập kiểm thử (591 mẫu)	39
Bảng 4.4: Phân loại các dạng lỗi chính của PhoBERT Fine-tuned.....	40
Bảng 5.1: Bốn chế độ xử lý câu hỏi của backend API.....	43

DANH MỤC HÌNH ẢNH

Hình 2.1: Pipeline tiền xử lý văn bản tiếng Việt.....	14
Hình 3.1: Code minh họa - Xây dựng và đánh giá TF-IDF Baseline	19
Hình 3.2: Code minh họa - Hàm <code>llm_generate_answer()</code> (RAG với Ollama API) .	20
Hình 3.3: Kiến trúc tổng quan hệ thống Hybrid FAQ + Generative RAG	21
Hình 3.4: Pipeline xây dựng index vector (offline)	23
Hình 3.5: Code minh họa - <code>build_index()</code> và <code>search_vector()</code> với ChromaDB.....	24
Hình 3.6: Code minh họa - Chia dữ liệu theo nhóm <code>faq_id</code> (<code>GroupShuffleSplit</code>) ...	25
Hình 3.7: Code minh họa - Lớp <code>FinetunedEncoder</code> (PhoBERT + <code>ViTokenizer</code>)	26
Hình 3.8: Công thức toán học <code>MultipleNegativesRankingLoss</code> và cách triển khai.	27
Hình 3.9: Công thức toán học LoRA và cấu hình thực tế ($r=16$, $\alpha=16$, 0.53% tham số)	30
Hình 4.1: Ma trận phân bố rank – Số lượng truy vấn theo vị trí FAQ đúng trong kết quả top-5	34
Hình 4.2: Biểu đồ so sánh các tiêu chí đánh giá giữa ba phương pháp	35
Hình 4.3: Log so sánh TF-IDF vs PhoBERT gốc	36
Hình 4.4: Đường cong loss trong quá trình huấn luyện PhoBERT Fine-tuned	37
Hình 4.5: Đường cong loss trong quá trình fine-tune Qwen2.5-7B với LoRA	38
Hình 4.6: Lịch trình learning rate (warmup + cosine decay) trong quá trình fine-tune Qwen2.5-7B.....	39
Hình 5.1: Giao diện chính của hệ thống.....	43
Hình 5.2: Kịch bản 1 – Mode 1 (Chỉ Retrieval), câu hỏi trong phạm vi	44
Hình 5.3: Kịch bản 2 – Mode 2 (RAG), tổng hợp từ top-K FAQ thông qua Qwen3.5:9b.....	45
Hình 5.4: Kịch bản 3 – Mode 3 (Hybrid), fallback sang Qwen2.5-7B LoRA khi retrieval dưới ngưỡng	46
Hình 5.5: Kịch bản 4 – Mode 1	47
Hình 5.6: Kịch bản 4 – Mode 2	47
Hình 5.7: Kịch bản 4 – Mode 3	47

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài

Trong môi trường đại học, sinh viên thường xuyên có nhu cầu tra cứu thông tin liên quan đến các vấn đề như tuyển sinh, đăng ký môn học, học phí, chương trình đào tạo, lịch học, cơ sở vật chất cũng như các quy định học vụ. Tuy nhiên, hiện nay phần lớn các câu hỏi này vẫn được giải quyết thông qua các phương thức truyền thống dẫn đến tình trạng quá tải và làm giảm tốc độ phản hồi.

Xuất phát từ thực tiễn đó, việc xây dựng một hệ thống hỏi đáp tự động dựa trên tập câu hỏi thường gặp là một giải pháp cần thiết. Hệ thống này cho phép cung cấp thông tin nhanh chóng, liên tục, góp phần nâng cao trải nghiệm của sinh viên đồng thời giảm áp lực cho bộ phận hỗ trợ của nhà trường.

Về mặt ý, hệ thống không chỉ hỗ trợ sinh viên tiếp cận thông tin một cách nhanh chóng và thuận tiện mà còn góp phần nâng cao tính chuyên nghiệp, hiện đại hóa công tác quản lý và phục vụ sinh viên. Đồng thời, hệ thống có khả năng mở rộng linh hoạt, cho phép bổ sung dữ liệu FAQ và cải tiến mô hình theo thời gian.

Cuối cùng, đề tài có tính khả thi cao nhờ vào sự sẵn có của nguồn dữ liệu FAQ từ các kênh chính thức của nhà trường, cũng như sự hỗ trợ của các thư viện và công cụ phát triển NLP hiện đại như PyTorch, Transformers và PhoBERT. Tóm lại, đề tài được lựa chọn dựa trên sự kết hợp giữa nhu cầu thực tiễn, giá trị ứng dụng cao và tính khả thi trong triển khai, đồng thời phù hợp với định hướng nghiên cứu của môn học.

2. Mục tiêu đề tài

Xây dựng một hệ thống hỏi đáp FAQ dành cho sinh viên, có khả năng tiếp nhận câu hỏi tiếng Việt và trả về câu trả lời phù hợp từ dữ liệu có sẵn.

3. Yêu cầu đặt ra

- Về độ chính xác: Đảm bảo hệ thống trả về câu trả lời phù hợp với các câu hỏi khác nhau để thực hiện đúng yêu cầu của người dùng, hạn chế tối đa tình trạng trả lời sai gây bất tiện cho người dùng.

- Tốc độ xử lý: Hệ thống phải trả lời một cách nhanh chóng, không được quá chậm trễ ảnh hưởng đến trải nghiệm người dùng.

4. Phạm vi và đối tượng

Phạm vi:

Dữ liệu giới hạn trong Sổ tay sinh viên 2025 đăng tải ngày 03/10/2025 được tổng hợp thành các cặp answer - question.

Xử lý văn bản tiếng Việt.

Đối tượng:

Đề án thực hiện dựa trên các phương pháp trong Xử lý ngôn ngữ tự nhiên, mô hình biểu diễn ngôn ngữ tiếng Việt (PhoBERT), các kỹ thuật truy xuất thông tin và đo độ tương đồng. Về phần mềm, chương trình huấn luyện mô hình được viết bằng ngôn ngữ Python trên Visual Studio Code IDE (kết hợp với cloud) với các thư viện hỗ trợ mạnh mẽ cho huấn luyện như Transformer, Torch, Pandas, PYVI...

5. Bố cục bài báo cáo

Bài báo cáo gồm 5 chương như sau:

- Chương 1: Cơ sở lý thuyết
- Chương 2: Mô tả bài toán và hệ thống dữ liệu
- Chương 3: Phương pháp đề xuất và kiến trúc hệ thống
- Chương 4: Thực nghiệm, kết quả và so sánh
- Chương 5: Demo ứng dụng

PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về Xử lý ngôn ngữ tự nhiên (Natural Language Processing)

1.1.1. Giới thiệu Xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP) là một lĩnh vực giao thoa giữa khoa học máy tính, trí tuệ nhân tạo (AI) và ngôn ngữ học. Mục tiêu của NLP là giúp máy tính có khả năng phân tích, hiểu và tạo ra ngôn ngữ của con người. Các ứng dụng phổ biến và đóng vai trò quan trọng trong NLP bao gồm dịch máy (machine translation), phân tích cảm xúc (sentiment analysis), tóm tắt văn bản (text summarization), nhận dạng thực thể có tên (NER), và các hệ thống hỏi đáp.

Việc xử lý ngôn ngữ tự nhiên đối mặt với nhiều thách thức lớn do sự mơ hồ của ngôn ngữ, cấu trúc ngữ pháp phức tạp và các đặc thù phân mảnh riêng biệt giữa các ngôn ngữ khác nhau trên thế giới. Để giải quyết các bài toán này, NLP hiện đại kết hợp giữa các quy tắc ngôn ngữ học truyền thống và các thuật toán Học máy (Machine Learning) hay Học sâu (Deep Learning).

1.1.2. Tokenization và Word Segmentation tiếng Việt (PyVi)

Tokenization (Tách token): Đây là bước tiền xử lý đầu tiên và cốt lõi trong bất kỳ hệ thống NLP nào. Tokenization là quá trình phân tách một chuỗi văn bản liên tục (running text) thành các đơn vị nhỏ hơn gọi là token. Tùy vào phương pháp, các token này có thể là từ (words), hình vị (morphemes), ký tự (characters) hoặc từ phụ (subwords). Việc chuẩn hóa văn bản thành các token giúp máy tính dễ dàng thống kê, phân tích và trích xuất đặc trưng.

Word Segmentation tiếng Việt: Đối với tiếng Anh, việc tách từ tương đối đơn giản vì ranh giới của từ thường được phân định bằng khoảng trắng. Tuy nhiên, đặc thù của tiếng Việt là một ngôn ngữ đơn lập, ranh giới khoảng trắng chỉ dùng để tách các "tiếng" (âm tiết) chứ không hoàn toàn đại diện cho một "từ" có nghĩa trọn vẹn (ví dụ: từ "học sinh", "ngôn ngữ" gồm nhiều âm tiết ghép lại). Nếu chỉ sử dụng khoảng trắng để tách, mô hình sẽ mất đi ý nghĩa gốc của từ ghép. Do đó, Word Segmentation (Tách từ) trong tiếng Việt là bài toán gom cụm các âm tiết lại thành một từ chuẩn xác.

Thư viện PyVi (Python Vietnamese Core NLP Toolkit): PyVi là một thư viện mã nguồn mở phổ biến trong Python, được thiết kế chuyên biệt để xử lý các tác vụ NLP cơ bản cho tiếng Việt. Thư viện này cung cấp module ViTokenizer sử dụng các thuật toán

tối ưu để xử lý bài toán tách từ tiếng Việt một cách tự động và chính xác. Bằng cách nhận diện đúng các từ ghép, PyVi tạo ra dữ liệu đầu vào chất lượng, giúp các mô hình phía sau hiểu đúng ngữ nghĩa của câu.

1.1.3. Text Embedding và Vectorization

Máy tính không thể trực tiếp xử lý các chuỗi ký tự văn bản, do đó dữ liệu chữ cần được chuyển đổi thành các con số hoặc biểu diễn toán học.

Vectorization: Là quá trình chuyển đổi các token thành các vector số. Các phương pháp cổ điển như Bag-of-Words (BoW) hay TF-IDF biểu diễn văn bản dựa trên tần suất đếm của từ. Tuy nhiên, nhược điểm của các phương pháp này là tạo ra các vector cực kỳ thưa thớt (sparse vectors), số chiều rất lớn và hoàn toàn bỏ qua thứ tự cũng như ý nghĩa ngữ nghĩa của từ.

Text Embedding (Nhúng văn bản): Để khắc phục các hạn chế trên, Word Embeddings ra đời. Đây là phương pháp biểu diễn từ dưới dạng các vector số thực liên tục trong một không gian đa chiều (dense vectors). Text embedding dựa trên giả thuyết phân bố trong ngôn ngữ học, cho rằng các từ xuất hiện trong những ngữ cảnh giống nhau thường có ý nghĩa tương đồng nhau.

Các mô hình phổ biến trong Text Embedding bao gồm Word2Vec và GloVe. Thuật toán Skip-gram, chẳng hạn, huấn luyện một mạng nơ-ron phân loại để phân biệt các từ nằm gần một từ mục tiêu so với các từ nhiễu ngẫu nhiên, từ đó học được trọng số chính là các vector nhúng. Những vector này rất hiệu quả vì chúng thu gọn số chiều dữ liệu, đồng thời bảo toàn được sự tương đồng về mặt ngữ nghĩa và cấu trúc ngữ pháp giữa các từ. Trong các tiến bộ NLP gần đây, các khái niệm này được mở rộng thành Contextual Embeddings (Nhúng theo ngữ cảnh) thông qua kiến trúc mạng Transformer, nơi biểu diễn của một từ sẽ linh hoạt thay đổi tùy vào toàn bộ câu chứa nó.

1.2. Các thuật toán và mô hình liên quan

1.2.1. TF-IDF và Cosine Similarity

Trước khi các mô hình Học sâu (Deep Learning) trở nên phổ biến, các phương pháp dựa trên tần suất từ khóa thường được sử dụng và hiện nay vẫn đóng vai trò là mô hình cơ sở hiệu quả, dễ cài đặt cho các bài toán phân loại hay tìm kiếm văn bản.

TF-IDF (Term Frequency - Inverse Document Frequency): là một kỹ thuật vector hóa nhằm đánh giá mức độ quan trọng của một từ đối với một tài liệu cụ thể trong một tập hợp tài liệu. Trọng số TF-IDF được tính bằng tích của hai thành phần:

TF (Term Frequency): Tần suất xuất hiện của từ t trong tài liệu d . Các từ xuất hiện nhiều trong một tài liệu sẽ có TF cao.

IDF (Inverse Document Frequency): Nghịch đảo tần suất tài liệu chứa từ t . Thành phần này giúp giảm trọng số của các từ quá phổ biến (như "và", "là", "của") và tăng trọng số cho các từ hiếm, mang nhiều thông tin đặc trưng.

Công thức tổng quát: $TF - IDF(t, d) = TF(t, d) \times IDF(t)$

Cosine Similarity (Độ tương đồng Cosine): Khi các văn bản đã được biểu diễn dưới dạng vector TF-IDF, ta cần một độ đo để tính toán mức độ giống nhau giữa chúng. Cosine Similarity đo lường góc giữa hai vector trong không gian đa chiều, thay vì đo khoảng cách hình học (Euclidean distance).

Công thức: $Cosine(A, B) = \frac{A \cdot B}{||A|| \times ||B||}$

Giá trị Cosine Similarity dao động từ 0 - 1, trong đó 1 chỉ ra rằng hai vector hoàn toàn cùng hướng (văn bản có ngữ nghĩa/từ vựng giống nhau) và 0 nghĩa là hai vector vuông góc (không có từ nào chung). Sự kết hợp giữa TF-IDF và Cosine Similarity tạo ra một hệ thống đối sánh văn bản nhanh chóng và ít tốn tài nguyên tính toán.

1.2.2. Mô hình ngôn ngữ PhoBERT

Khi các mô hình biểu diễn từ theo ngữ cảnh (Contextual Word Embeddings) ra đời, đặc biệt là kiến trúc Transformer, chất lượng của các hệ thống NLP đã tăng vọt. Đối với tiếng Việt, PhoBERT là một trong những mô hình ngôn ngữ tiền huấn luyện (pre-trained language model) nổi bật nhất.

Kiến trúc và Phương pháp: PhoBERT được xây dựng dựa trên kiến trúc RoBERTa (A Robustly Optimized BERT Pretraining Approach), một phiên bản cải tiến tối ưu của BERT. RoBERTa loại bỏ tác vụ Dự đoán câu tiếp theo của BERT và sử dụng kỹ thuật Che mặt nạ động (Dynamic Masking), giúp mô hình học các biểu diễn ngữ nghĩa linh hoạt và sâu sắc hơn.

Đặc điểm dành riêng cho tiếng Việt: Khác với các mô hình đa ngôn ngữ thường chia cắt từ tiếng Việt một cách tùy ý thành các subword (từ phụ), PhoBERT được tiền huấn luyện trên khối lượng dữ liệu khổng lồ với mức độ "từ". Văn bản đầu vào của PhoBERT thường được tách từ (Word Segmentation) bằng các công cụ như VnCoreNLP hay PyVi trước khi đưa vào mô hình. Nhờ vậy, PhoBERT có khả năng nắm bắt ngữ pháp và các đặc trưng ngôn ngữ đơn lập của tiếng Việt vượt trội hơn hẳn.

Ứng dụng: Trong các bài toán học máy phân tích ngôn ngữ tự nhiên cho tiếng Việt như Phân loại cảm xúc, Nhận dạng thực thể có tên, hay Suy luận tự nhiên (NLI), PhoBERT thường được sử dụng làm bộ trích xuất đặc trưng cực kỳ mạnh mẽ.

1.2.3. Mô hình ngôn ngữ lớn (LLM) và họ Qwen

Sự tiến hóa của NLP hiện nay đang xoay quanh các Mô hình ngôn ngữ lớn (Large Language Models - LLMs) với hàng tỷ đến hàng trăm tỷ tham số.

Kiến trúc của LLM: Phần lớn các LLM hiện đại (như họ GPT) đều sử dụng kiến trúc Transformer Decoder-only (chỉ có bộ giải mã). Thay vì học hai chiều như Masked Language Models (PhoBERT/BERT), LLMs sử dụng phương pháp Causal Language Modeling (hay Autoregressive LMs) – tức là dự đoán xác suất của từ tiếp theo dựa trên chuỗi các từ đã xuất hiện trước đó từ trái sang phải. Nhờ được huấn luyện trên kho dữ liệu khổng lồ bao phủ nhiều lĩnh vực, LLM có khả năng "in-context learning" (học theo ngữ cảnh), cho phép chúng giải quyết các tác vụ mới mà không cần phải tinh chỉnh (fine-tuning) quá nhiều thông qua các prompt.

Họ mô hình Qwen: Qwen (Qwen1.5, Qwen2,...) là một họ các mô hình ngôn ngữ lớn mã nguồn mở do Alibaba Cloud phát triển. Đây là một trong những đại diện xuất sắc của các LLM thế hệ mới với nhiều ưu điểm:

Hỗ trợ đa ngôn ngữ mạnh mẽ: Qwen được huấn luyện trên tập dữ liệu đa dạng và tối ưu hóa tốt cho nhiều ngôn ngữ, bao gồm cả tiếng Việt. Điều này giúp mô hình sinh văn bản tiếng Việt tự nhiên, ít bị "lai" cấu trúc ngữ pháp nước ngoài so với các mô hình phương Tây giai đoạn đầu.

Khả năng tư duy và lập trình: Kiến trúc của Qwen tích hợp các cải tiến hiện đại như Grouped-Query Attention (GQA) để xử lý ngữ cảnh dài mượt mà, RoPE (Rotary Positional Embeddings) và SwiGLU activation. Nhờ đó, Qwen không chỉ giỏi sinh văn bản mà còn có năng lực suy luận logic (reasoning) và hiểu code rất tốt.

Tính linh hoạt: Họ Qwen cung cấp nhiều phiên bản kích thước khác nhau (từ vài tỷ tham số để chạy trên thiết bị cá nhân đến 72B tham số cho các hệ thống máy chủ lớn), giúp các nhà nghiên cứu dễ dàng chọn lựa mô hình phù hợp.

1.3. Bài toán QA (Question Answering)

1.3.1. Tổng quan hệ thống Hỏi đáp (QA)

Hỏi đáp (Question Answering - QA) là một trong những bài toán cốt lõi và mang tính ứng dụng cao nhất của Xử lý ngôn ngữ tự nhiên. Mục tiêu của một hệ thống QA là tự động trả lời các câu hỏi do con người đặt ra bằng ngôn ngữ tự nhiên.

Dựa vào nguồn tri thức và phương pháp xử lý, hệ thống QA thường được chia thành hai dạng chính:

Hỏi đáp dựa trên tài liệu (Reading Comprehension/Closed-domain QA): Hệ thống được cung cấp một đoạn văn bản và một câu hỏi. Nhiệm vụ của mô hình là đọc hiểu đoạn văn và trích xuất hoặc tổng hợp câu trả lời trực tiếp từ đoạn văn đó.

Hỏi đáp miền mở (Open-domain QA): Hệ thống không được cung cấp sẵn một tài liệu cụ thể mà phải tự tìm kiếm thông tin từ một cơ sở tri thức lớn (như Wikipedia hoặc kho tài liệu của doanh nghiệp) để trả lời.

Hệ thống QA truyền thống thường dựa vào các kỹ thuật Trích xuất thông tin (Information Retrieval - IR) kết hợp với nhận dạng mẫu. Tuy nhiên, với sự phát triển của các Mô hình ngôn ngữ lớn (LLM), các hệ thống QA hiện đại đã chuyển sang phương pháp sinh văn bản (Generative QA), nơi mô hình không chỉ trích xuất mà còn có khả năng tự tổng hợp, tóm tắt và diễn đạt lại câu trả lời một cách tự nhiên.

1.3.2. Kiến trúc Retrieval-Augmented Generation (RAG)

Mặc dù các LLM có khả năng sinh văn bản xuất sắc, chúng vẫn gặp phải những hạn chế lớn khi áp dụng vào bài toán QA thực tế: hiện tượng "ảo giác" (hallucination - sinh ra thông tin sai lệch nhưng với giọng điệu tự tin), kiến thức bị lỗi thời và thiếu khả năng truy cập vào dữ liệu nội bộ/bảo mật của người dùng.

Để giải quyết triệt để các vấn đề này, kiến trúc Retrieval-Augmented Generation (RAG) đã được giới thiệu và nhanh chóng trở thành tiêu chuẩn trong việc xây dựng hệ thống QA hiện đại. RAG kết hợp giữa hai thành phần chính:

Bộ truy xuất (Retriever): Khi có câu hỏi, Retriever sẽ tìm kiếm và trích xuất các đoạn văn bản có chứa thông tin liên quan nhất từ một cơ sở tri thức bên ngoài.

Bộ sinh (Generator): Các đoạn văn bản tìm được sẽ được ghép nối với câu hỏi ban đầu để tạo thành một câu lệnh hoàn chỉnh. LLM sau đó sẽ đóng vai trò là Generator, đọc ngữ cảnh được cung cấp và sinh ra câu trả lời chính xác, có căn cứ.

Ưu điểm vượt trội của RAG: RAG giúp hệ thống cung cấp thông tin cập nhật, chính xác và có thể trích dẫn nguồn gốc mà không cần phải tốn chi phí và tài nguyên để tinh chỉnh (fine-tune) lại toàn bộ mô hình ngôn ngữ khổng lồ.

1.3.3. Cơ sở dữ liệu Vector (ChromaDB)

Để bộ truy xuất hoạt động với tốc độ cao và độ chính xác tốt trên tập dữ liệu lớn, ta không thể tìm kiếm theo từ khóa thông thường mà phải tìm kiếm theo ngữ nghĩa. Đây là lúc Cơ sở dữ liệu Vector (Vector Database) phát huy tác dụng.

Cơ sở dữ liệu Vector: Khác với cơ sở dữ liệu quan hệ lưu trữ dữ liệu dạng bảng, Vector Database lưu trữ dữ liệu dưới dạng các vector nhúng (embeddings) mang nhiều chiều. Dữ liệu văn bản (tài liệu, bài viết) trước tiên sẽ được chia nhỏ (chunking) và đưa qua một mô hình Text Embedding (như đã trình bày ở mục 1.1.3) để chuyển thành vector. Khi có một truy vấn từ người dùng, truy vấn này cũng được chuyển thành vector tương ứng. Vector Database sau đó sẽ thực hiện phép tính độ tương đồng (thường dùng Cosine Similarity hoặc tích vô hướng) để tìm ra các vector tài liệu nằm gần nhất với vector truy vấn trong không gian đa chiều.

ChromaDB: ChromaDB là một cơ sở dữ liệu vector mã nguồn mở được thiết kế chuyên biệt để xây dựng các ứng dụng AI kết hợp với LLM.

Đặc điểm: ChromaDB rất nhẹ, dễ cài đặt và tối ưu hóa tốt cho việc lưu trữ cả văn bản thô (documents), siêu dữ liệu (metadata) và vector nhúng (embeddings).

Tích hợp: Nó cung cấp API thân thiện với Python và tích hợp sâu mẽ với các framework phát triển LLM như LangChain hay LlamaIndex.

Trong một quy trình RAG thông thường, ChromaDB sẽ đóng vai trò là kho lưu trữ bộ nhớ dài hạn, giúp truy xuất ngữ cảnh ngữ nghĩa cực kỳ nhanh chóng để cung cấp cho bộ sinh LLM, từ đó hoàn thiện chu trình của hệ thống hỏi đáp.

1.4. Các thư viện sử dụng

1.4.1. Nhóm thư viện học sâu và xử lý mô hình ngôn ngữ

PyTorch (torch) là framework học sâu nền tảng của toàn bộ hệ thống. Thư viện này cung cấp kiến trúc tính toán tensor tối ưu hóa cho GPU CUDA, cho phép thực hiện các phép tính ma trận quy mô lớn với tốc độ cao - điều kiện thiết yếu khi huấn luyện và suy diễn với các mô hình Transformer hàng triệu đến hàng tỷ tham số. Trong đồ án này, PyTorch 2.0+ kết hợp với CUDA 12.1 được sử dụng xuyên suốt từ giai đoạn fine-tune PhoBERT đến quá trình suy diễn của mô hình sinh Qwen2.5-7B.

Transformers (HuggingFace) là thư viện trung tâm cung cấp các lớp trừu tượng để nạp, cấu hình và sử dụng hàng nghìn mô hình ngôn ngữ tiền huấn luyện. Trong đồ án, thư viện này được dùng để nạp mô hình `vinai/phobert-base-v2` và các tokenizer tương ứng, đồng thời hỗ trợ quá trình fine-tune mô hình sinh `Qwen2.5-7B-Instruct` thông qua lớp `AutoModelForCausalLM` và `AutoTokenizer`. Phiên bản 4.40.x được sử dụng, đảm bảo tương thích đầy đủ với kiến trúc `Qwen2.5` và các kỹ thuật quantization 4-bit.

Sentence-Transformers là framework chuyên biệt cho bài toán tạo sentence embedding và học biểu diễn ngữ nghĩa. Thư viện này đóng gói kiến trúc `SentenceTransformer` (bao gồm bộ encoder và lớp pooling), đồng thời cung cấp các hàm mất mát phục vụ học đối sánh như `MultipleNegativesRankingLoss` - hàm loss cốt lõi trong quá trình fine-tune `PhoBERT` của đồ án. Ngoài ra, thư viện còn hỗ trợ các lớp đánh giá như `BinaryClassificationEvaluator` và lớp `SentenceTransformerTrainer` để tự động hóa vòng lặp huấn luyện.

PEFT (Parameter-Efficient Fine-Tuning) là thư viện của HuggingFace cung cấp các kỹ thuật tinh chỉnh mô hình hiệu quả tham số, trong đó nổi bật nhất là `LoRA` (Low-Rank Adaptation). Thay vì cập nhật toàn bộ trọng số của mô hình ngôn ngữ lớn, PEFT chỉ huấn luyện một tập con nhỏ các ma trận rank thấp được chèn vào các lớp attention, qua đó giảm thiểu đáng kể yêu cầu VRAM và thời gian huấn luyện. Thư viện này được sử dụng kết hợp với `Unsloth` để tinh chỉnh `Qwen2.5-7B-Instruct` trên GPU `T4 16GB`, với chỉ khoảng 0,53% tham số được cập nhật trong suốt quá trình fine-tune.

`BitsAndBytes` là thư viện cho phép nạp và chạy suy diễn các mô hình ngôn ngữ lớn dưới dạng quantization 4-bit (NF4) hoặc 8-bit, giúp giảm dung lượng bộ nhớ GPU cần thiết gần bốn lần so với định dạng `float16` mặc định. Điều này cho phép nạp và suy diễn mô hình `Qwen2.5-7B` trên GPU `NVIDIA T4 16GB` mà không vượt quá ngưỡng VRAM. `Accelerate` là thư viện hỗ trợ của HuggingFace, trừu tượng hóa việc phân phối tính toán trên nhiều thiết bị (single GPU, multi-GPU, CPU offloading), đảm bảo pipeline huấn luyện hoạt động ổn định và có thể mở rộng.

1.4.2. Nhóm thư viện xử lý dữ liệu và NLP tiếng Việt

`PyVi` (`pyvi`) là bộ công cụ NLP cốt lõi cho tiếng Việt trong đồ án này. Như đã trình bày tại mục 1.1.2, module `ViTokenizer` của `PyVi` thực hiện bài toán tách từ (word segmentation) tiếng Việt, chuyển đổi chuỗi âm tiết thành các từ ghép có nghĩa trước khi

đưa vào mô hình PhoBERT. Việc áp dụng tách từ nhất quán trên cả dữ liệu huấn luyện lẫn câu hỏi truy vấn tại thời điểm suy diễn là điều kiện bắt buộc để đảm bảo tính nhất quán của không gian embedding.

Pandas là thư viện xử lý dữ liệu dạng bảng, đóng vai trò là công cụ chính để đọc, lọc, biến đổi và lưu trữ tập dữ liệu FAQ dưới định dạng CSV và JSONL. Mọi thao tác trên dữ liệu - từ đọc file FAQ gốc, sinh và lọc paraphrase, đến tổng hợp thống kê phân bố dữ liệu - đều được thực hiện thông qua DataFrame API của Pandas. NumPy hỗ trợ Pandas bằng cách cung cấp các phép tính mảng số đa chiều hiệu quả, đặc biệt hữu ích trong giai đoạn tính toán Cosine Similarity và các chỉ số đánh giá Precision@K trên tập vector embedding.

Datasets (HuggingFace) cung cấp cơ sở hạ tầng để tải, xử lý và quản lý tập dữ liệu theo chuẩn định dạng Arrow, tích hợp liền mạch với SentenceTransformerTrainer và SFTTrainer. Thư viện này cho phép xây dựng pipeline dữ liệu hiệu quả với bộ nhớ thấp thông qua cơ chế lazy loading, đặc biệt quan trọng khi xử lý tập paraphrase hơn 10.000 mẫu trên môi trường có RAM hạn chế. Scikit-learn bổ sung các công cụ machine learning truyền thống, trong đó quan trọng nhất là TfidfVectorizer - được dùng để xây dựng mô hình TF-IDF baseline trong các thực nghiệm đánh giá so sánh ở Chương 4.

1.4.3. Nhóm thư viện cơ sở hạ tầng và triển khai ứng dụng

ChromaDB, như đã trình bày chi tiết tại mục 1.3.3, là cơ sở dữ liệu vector mã nguồn mở đảm nhận toàn bộ chức năng lưu trữ và tìm kiếm ngữ nghĩa trong hệ thống. Phiên bản 0.5.x sử dụng PersistentClient giúp dữ liệu vector được lưu trữ bền vững trên ổ đĩa, tránh phải tái tạo lại chỉ mục mỗi lần khởi động ứng dụng. Cấu trúc tìm kiếm HNSW (Hierarchical Navigable Small World) với metric cosine đảm bảo tốc độ truy vấn gần đúng logarithm theo kích thước tập dữ liệu, phù hợp với yêu cầu.

Flask là micro-framework Python cho phát triển web API, được lựa chọn làm nền tảng backend của ứng dụng demo vì tính nhẹ nhàng và dễ tích hợp với các thư viện học sâu. Flask phục vụ hai endpoint chính (/ask và /chat) xử lý các yêu cầu POST từ giao diện frontend, đồng thời quản lý vòng đời của các tài nguyên mô hình (PhoBERT, ChromaDB, Qwen2.5+LoRA) trong bộ nhớ ứng dụng. Requests là thư viện HTTP client đơn giản, được dùng trong pipeline sinh paraphrase để giao tiếp với Ollama API cục bộ thông qua các yêu cầu POST JSON.

CHƯƠNG 2: MÔ TẢ BÀI TOÁN VÀ HỆ THỐNG DỮ LIỆU

2.1. Phát biểu bài toán

2.1.1. Định nghĩa bài toán

Bài toán được thuộc lĩnh vực Hỏi-Đáp (Question Answering - QA), hỗ trợ sinh viên tra cứu thông tin từ hệ thống câu hỏi thường gặp (FAQ) của Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh (HCMUTE).

Bài toán có thể được định nghĩa như sau: cho trước một kho FAQ gồm N cặp (câu hỏi, câu trả lời) đã được biên soạn sẵn, hệ thống cần tự động tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên từ người dùng và trả về câu trả lời phù hợp nhất. Điểm đặc biệt của bài toán này so với các bài toán hỏi-đáp mở là phạm vi tri thức được giới hạn trong tập FAQ đã cho, do đó đòi hỏi khả năng so khớp ngữ nghĩa (semantic matching) chính xác giữa câu hỏi người dùng nhập vào và các câu hỏi trong kho FAQ, bất kể người dùng diễn đạt theo cách nào (informal, viết tắt, lỗi chính tả,...).

Thách thức chính của bài toán nằm ở đặc điểm của ngôn ngữ tiếng Việt: ngôn ngữ đơn lập, giàu biến thể từ vựng và cấu trúc câu linh hoạt. Một câu hỏi duy nhất trong FAQ có thể được người dùng diễn đạt theo hàng chục cách khác nhau, từ trang trọng đến thông thường của sinh viên trong đời sống. Các phương pháp truyền thống dựa trên so khớp từ khóa hay TF-IDF tỏ ra không đủ mạnh trong tình huống này, đặt ra nhu cầu sử dụng các mô hình biểu diễn ngữ nghĩa dày đặc dựa trên Transformer.

Bên cạnh đó, hệ thống còn phải xử lý trường hợp câu hỏi vượt phạm vi: khi câu hỏi người dùng không có tương quan ngữ nghĩa đủ mạnh với bất kỳ mục FAQ nào, hệ thống cần nhận biết và phản hồi phù hợp thay vì cưỡng ép trả lời sai. Để giải quyết vấn đề này, một cơ chế phân luồng dựa trên ngưỡng độ tương đồng được tích hợp vào kiến trúc hệ thống, kết hợp với mô hình sinh ngôn ngữ để xử lý các trường hợp này.

2.1.2. Xác định Input và Output

Bài toán được hình thức hóa với các thành phần input và output như sau:

Thành phần	Mô tả
Input chính	Câu hỏi bằng tiếng Việt do người dùng nhập vào: văn bản tự nhiên, có thể chứa lỗi chính tả, viết tắt, emoji, ký tự đặc biệt.
Output	Câu trả lời bằng tiếng Việt.

Bảng 2.1: Xác định Input và Output của bài toán

Cụ thể, quá trình xử lý một câu hỏi đầu vào trải qua ba giai đoạn. Đầu tiên là tiền xử lý và chuẩn hóa câu hỏi. Tiếp theo là truy xuất FAQ liên quan thông qua so sánh vector embedding. Tiếp theo là phân luồng và sinh câu trả lời dựa trên độ tương đồng và chế độ hoạt động được chọn.

2.2. Thu thập và phân tích dữ liệu

2.2.1. Nguồn dữ liệu

Dữ liệu được sử dụng là tập FAQ của Trường Đại học Công nghệ Kỹ thuật TP.HCM (HCMUTE), được thu thập và biên soạn từ nhiều nguồn chính thống của nhà trường, cụ thể là từ Sổ tay sinh viên 2025 (đăng tải ngày 03/10/2025).

Tập dữ liệu được lưu trữ dưới định dạng CSV, bao gồm ba cột chính: `faq_id` (định danh duy nhất cho mỗi mục FAQ), `question` (câu hỏi chuẩn), và `answer` (câu trả lời tương ứng). Mỗi mục FAQ đại diện cho một câu hỏi độc lập trong phạm vi thông tin học thuật và hành chính của trường.

Nội dung FAQ bao phủ nhiều chủ đề quan trọng liên quan đến đời sống học thuật của sinh viên, bao gồm: quy chế học tập và thi cử, thủ tục hành chính, hoạt động ngoại khóa, học bổng và hỗ trợ tài chính, cơ sở vật chất và dịch vụ sinh viên, và các thông tin chung về trường. Sự đa dạng về chủ đề đặt ra yêu cầu cao đối với khả năng hiểu ngữ nghĩa của mô hình truy xuất.

Quá trình thu thập và tổng dữ liệu được thực hiện với sự kiểm tra kỹ lưỡng về tính chính xác của nội dung, đảm bảo mỗi câu trả lời phản ánh đúng chính sách và quy định hiện hành của nhà trường tại thời điểm thu thập.

2.2.2. Đặc điểm ngôn ngữ, số lượng mẫu và lớp

Tập dữ liệu mang đặc trưng ngôn ngữ tiếng Việt đặc thù trong môi trường học thuật đại học. Câu hỏi trong FAQ thường có cấu trúc đơn giản, dạng câu hỏi trực tiếp (ai, gì, như thế nào, ở đâu, khi nào), có thể chứa thuật ngữ chuyên ngành giáo dục đại học và các từ viết tắt phổ biến (HCMUTE, TP.HCM,...).

Sau quá trình thu thập, tập dữ liệu gốc được mở rộng thông qua kỹ thuật tăng cường dữ liệu (data augmentation) bằng mô hình ngôn ngữ lớn. Mỗi câu hỏi gốc được sinh ra từ 5 đến 6 paraphrase, tạo thành tập dữ liệu huấn luyện cho bước fine-tune PhoBERT. Tập dữ liệu paraphrase đầy đủ đạt 10.502 cặp câu (`sentence1`: paraphrase, `sentence2`: câu hỏi gốc), được chia theo tỷ lệ 80% train, 10% validation, 10% test với chiến lược chia theo nhóm `faq_id` để tránh rò rỉ dữ liệu (data leakage) giữa các tập.

Thành phần dữ liệu	Số lượng	Ghi chú
FAQ gốc	~1148 mục	Data ban đầu (câu hỏi - trả lời)
Cặp paraphrase tổng	10.502 cặp	sentence1 (paraphrase); sentence2 (gốc)
Tập train	~80%	Chia theo faq_id group
Tập validation	~10%	BinaryClassificationEvaluator
Tập test	~10%	Đánh giá Precision@K, Cosine sim

Bảng 2.2: Thống kê số lượng mẫu dữ liệu

Mỗi nhóm FAQ (faq_id) chỉ xuất hiện trong đúng một trong ba tập, đảm bảo rằng quá trình đánh giá phản ánh trung thực khả năng tổng quát hóa của mô hình với các mục FAQ chưa từng gặp trong quá trình huấn luyện. Đây là điều kiện thiết yếu để tránh việc mô hình ghi nhớ thay vì học hiểu ngữ nghĩa.

Về bản chất bài toán, đây không phải là bài toán phân loại đa lớp truyền thống mà là bài toán retrieval theo thứ hạng. Với mỗi câu hỏi truy vấn, mô hình cần xếp hạng tất cả N câu hỏi trong FAQ theo thứ tự tương đồng ngữ nghĩa, với kỳ vọng câu hỏi đúng xuất hiện ở vị trí hàng đầu.

2.2.3. Phân tích phân bố dữ liệu

Câu hỏi gốc có độ dài từ 8 đến 25 từ (sau word segmentation), phù hợp với phong cách hỏi thẳng vào vấn đề trong bối cảnh FAQ. Một số câu hỏi phức tạp hơn có thể dài đến 40 – 50 từ, trong khi câu ngắn nhất chỉ có 4 – 5 từ.

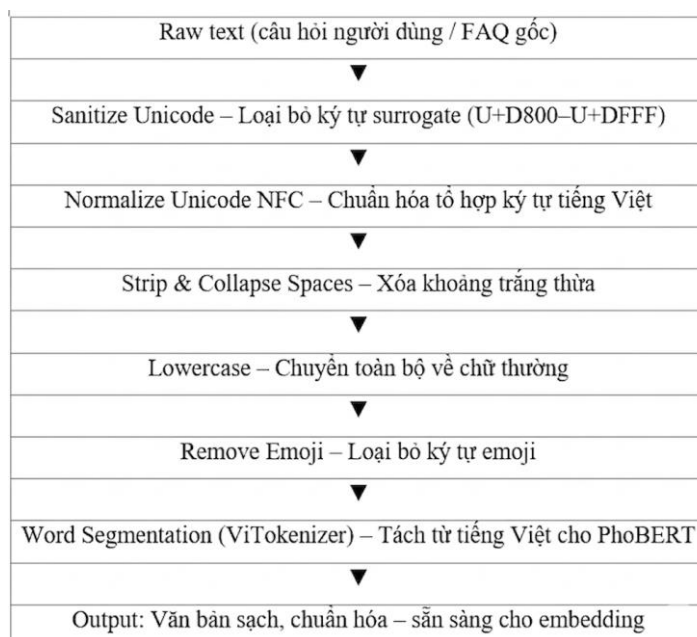
Về phía paraphrase do LLM sinh, độ dài câu có xu hướng biến thiên nhiều hơn so với câu gốc. Kỹ thuật sinh paraphrase thông thường cho ra các câu có độ dài tương đương hoặc ngắn hơn câu gốc, trong khi kỹ thuật sinh paraphrase mạnh tạo ra các diễn đạt có cấu trúc câu thay đổi nhiều hơn, đôi khi dài hơn do thêm thành phần bổ ngữ.

Về từ khóa và phân bố chủ đề, phân tích tần suất xuất hiện cho thấy một số từ/cụm từ xuất hiện nhiều trong FAQ như: "sinh viên", "trường", "HCMUTE",...

Đặc biệt, phân tích so sánh TF-IDF giữa paraphrase thông thường và paraphrase mạnh cho thấy sự khác biệt đáng kể: paraphrase thông thường giữ lại khoảng 60 – 70% từ khóa quan trọng của câu gốc, trong khi paraphrase mạnh chỉ giữ lại 30 – 40%. Điều này được thiết kế có chủ đích, giúp so sánh PhoBERT so với TF-IDF.

2.3. Tiền xử lý dữ liệu

Quá trình tiền xử lý dữ liệu đóng vai trò nền tảng trong pipeline, đảm bảo tính nhất quán giữa câu hỏi FAQ trong data và câu hỏi người dùng nhập vào.



Hình 2.1: Pipeline tiền xử lý văn bản tiếng Việt

2.3.1. Sanitize Unicode và chuẩn hóa NFC

Bước đầu tiên trong pipeline tiền xử lý là loại bỏ các ký tự Unicode không hợp lệ, đặc biệt là các ký tự surrogate trong dải U+D800 đến U+DFFF. Các ký tự này không thể encode hoặc decode theo chuẩn UTF-8 thông thường và thường xuất hiện do quá trình copy từ một số nguồn dữ liệu, hoặc trong kết quả sinh ra từ các LLM. Nếu không được xử lý, có thể gây ra lỗi nghiêm trọng trong quá trình xử lý chuỗi và embedding.

Sau bước sanitize (làm sạch), văn bản được chuẩn hóa về dạng Unicode NFC. Đây là bước quan trọng đối với tiếng Việt, vì cùng một ký tự có dấu có thể được biểu diễn bằng nhiều cách khác nhau trong Unicode: dạng kết hợp precomposed ví dụ ế = U+1EBF hoặc dạng decomposed ví dụ ế = e + ^ + ' (U+0065 + U+0302 + U+0301). Nếu không chuẩn hóa, hai chuỗi trông giống nhau về mặt hiển thị nhưng khác biệt về mặt biểu diễn byte sẽ bị coi là khác nhau khi so sánh chuỗi hoặc khi tokenize.

Input: text (chuỗi ký tự thô)
Output: text_clean (chuỗi đã làm sạch)
function sanitize_and_normalize(text):
1. IF text is None OR not string → RETURN ""

2. // Loại bỏ ký tự surrogate
text ← JOIN [c for c in text IF ord(c) NOT IN [0xD800, 0xDFFF]]
3. // Chuẩn hóa Unicode về dạng NFC
text ← unicodedata.normalize("NFC", text)
4. RETURN text

Thuật toán 2.1: Sanitize Unicode và chuẩn hóa NFC

2.3.2. Làm sạch văn bản

Sau bước chuẩn hóa Unicode, pipeline tiếp tục với ba bước làm sạch văn bản: chuẩn hóa khoảng trắng, chuyển về chữ thường và loại bỏ emoji.

Chuẩn hóa khoảng trắng được thực hiện thông qua hai thao tác: loại bỏ khoảng trắng ở đầu và cuối chuỗi, đồng thời gộp các khoảng trắng liên tiếp thành một khoảng trắng duy nhất bằng biểu thức chính quy `\s+`. Bước này giúp đảm bảo tính nhất quán của văn bản, đặc biệt đối với các dữ liệu thu thập từ nhiều nguồn khác nhau.

Bước chuyển về chữ thường (lowercase) giúp tăng recall khi so khớp, ví dụ "HCMUTE" và "hcmute" được coi là tương đương. Đặc biệt với tiếng Việt, việc viết hoa/thường thường không nhất quán trong văn bản không chính thức của sinh viên. Lưu ý rằng chỉ cột question được lowercase, cột answer giữ nguyên để hiển thị đẹp.

Bước loại bỏ emoji sử dụng biểu thức chính quy bao phủ các khối Unicode chứa emoji phổ biến: biểu cảm khuôn mặt (U+1F600–U+1F64F), biểu tượng và hình vẽ (U+1F300–U+1F5FF), phương tiện và bản đồ (U+1F680–U+1F6FF), cờ các quốc gia (U+1F1E0–U+1F1FF) và các ký tự đặc biệt khác. Emoji thường xuất hiện trong tin nhắn của sinh viên nhưng không có mặt trong FAQ gốc, nên chúng có thể gây nhiễu cho quá trình embedding nếu không được loại bỏ.

Input: text (chuỗi sau sanitize_and_normalize)
Output: text_clean (chuỗi sạch sẵn sàng cho embedding)
EMOJI_PATTERN ← RegEx covering U+1F600-1F64F, 1F300-1F5FF, 1F680-1F6FF, 1F1E0-1F1FF, 2702-27B0
function preprocess(text):
1. text ← sanitize_and_normalize(text) // Bước 2.3.1
2. text ← text.strip() // Xóa đầu/cuối
3. text ← regex.sub(r"\s+", " ", text) // Collapse spaces

4. text ← text.lower()	// Chữ thường
5. text ← EMOJI_PATTERN.sub("", text)	// Xóa emoji
6. text ← regex.sub(r"\s+", " ", text).strip()	// Dọn lại
7. RETURN text	

Thuật toán 2.2: Pipeline làm sạch văn bản đầy đủ

Nguồn	Gốc (ví dụ)	Sau preprocessing
FAQ	Trường Đại học Sư phạm Kỹ thuật TP. HCM có bao nhiêu sinh viên?	trường đại học sư phạm kỹ thuật tp. hcm có bao nhiêu sinh viên?
FAQ	Việc chọn học tại HCMUTE có ý nghĩa như thế nào?	việc chọn học tại hcmute có ý nghĩa như thế nào?
Para.	Bức thư dành cho TÂN SINH VIÊN khóa 2025 do AI viết? 😊	bức thư dành cho tân sinh viên khóa 2025 do ai viết?

Bảng 2.3: Mẫu kết quả tiền xử lý dữ liệu

2.3.3. Kỹ thuật sinh dữ liệu Paraphrase

Do số lượng câu hỏi FAQ gốc còn hạn chế so với yêu cầu của việc fine-tune mô hình, kỹ thuật tăng cường dữ liệu (data augmentation) sinh paraphrase bằng LLM được áp dụng. Đây là bước để tạo ra tập dữ liệu huấn luyện đủ lớn và đa dạng cho PhoBERT.

Đồ án này sử dụng mô hình Qwen3.5:9B thông qua API Ollama chạy cục bộ để sinh paraphrase. Mô hình này được chọn vì có khả năng sinh tiếng Việt chất lượng cao, ổn định hơn các lựa chọn khác như deepseek-r1:7b. Mỗi câu hỏi gốc được lệnh sinh ra từ 5 đến 6 paraphrase. Sau khi lọc và kiểm tra chất lượng, lấy tối đa 5 paraphrase.

Hai loại paraphrase được sinh ra phục vụ các mục đích khác nhau. Paraphrase thông thường sinh các câu có cùng ý nghĩa với cách diễn đạt đa dạng (formal/informal, dài/ngắn), thường vẫn giữ một số từ khóa quan trọng từ câu gốc. Paraphrase mạnh yêu cầu mô hình tối thiểu hóa sự trùng lặp từ khóa, sử dụng từ đồng nghĩa và biến đổi cấu trúc câu sâu hơn.

Input: question (câu hỏi FAQ đã preprocess), n=5
Output: paraphrases (danh sách $\leq n$ paraphrase hợp lệ)
function generate_paraphrase(question, n, max_retries=3):
FOR attempt IN 1..max_retries:
1. Tạo prompt với question và yêu cầu sinh n+1 paraphrase

2. Gọi Ollama API (Qwen3.5:9b, temperature=0.7)
3. Parse response: tách dòng, xóa prefix số thứ tự
4. Lọc hợp lệ (is_valid_vietnamese):
- Độ dài ≥ 10 ký tự
- Không chứa ký tự CJK (Hán/Nhật/Hàn)
- Không chứa từ tiếng Anh/Đức phổ biến
- Không phải câu từ chối/lỗi
5. IF valid_lines not empty $\rightarrow$ RETURN valid_lines[:n]
6. ELSE $\rightarrow$ log retry, sleep(1s), continue
RETURN [] (fallback: dùng câu gốc làm positive pair)

Thuật toán 2.3: Sinh và lọc paraphrase bằng LLM

Kết quả đầu ra của quá trình sinh paraphrase được lưu dưới hai định dạng: JSONL và CSV. Mỗi dòng trong file JSONL chứa một cặp (sentence1: paraphrase, sentence2: câu hỏi gốc) đại diện cho một cặp câu tương đồng dương tính (positive pair) dùng trong thuật toán MultipleNegativesRankingLoss khi huấn luyện PhoBERT.

Sau khi sinh paraphrase, bước tiền xử lý cuối cùng được thực hiện trên sentence1 (paraphrase) bằng script, áp dụng cùng pipeline preprocessing như đã mô tả, nhằm đảm bảo sentence1 và sentence2 có cùng định dạng chuẩn hóa trước khi đưa vào huấn luyện.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT VÀ KIẾN TRÚC HỆ THỐNG

3.1. Tổng quan các phương pháp tiếp cận

Để giải quyết bài toán hỏi-đáp FAQ cho trường đại học, nhóm khảo sát và đánh giá ba phương pháp tiếp cận cơ bản trước khi đề xuất kiến trúc tích hợp. Mỗi phương pháp có ưu và nhược điểm riêng.

3.1.1. Baseline 1: FAQ Retrieval truyền thống

Phương pháp truyền thống nhất là so khớp từ khóa (keyword matching) hoặc TF-IDF. Trong phương pháp này, câu hỏi người dùng và các câu hỏi FAQ được biểu diễn dưới dạng vector thưa (sparse vector) dựa trên tần suất xuất hiện của từ, sau đó tính độ tương đồng cosine để tìm câu hỏi FAQ gần nhất.

Ưu điểm của phương pháp này là tốc độ nhanh, không đòi hỏi huấn luyện, và dễ triển khai. Tuy nhiên, nhược điểm cốt lõi là hoàn toàn phụ thuộc vào sự trùng lặp từ khóa: nếu người dùng dùng từ đồng nghĩa hoặc cách diễn đạt khác, TF-IDF sẽ cho điểm thấp dù ý nghĩa câu hỏi là giống nhau. Thực nghiệm trong nghiên cứu này cho thấy TF-IDF cho $P@1$ thấp hơn đáng kể so với PhoBERT fine-tuned, đặc biệt trên tập paraphrase mạnh.

Ngoài TF-IDF, phương pháp BM25 cải tiến TF-IDF bằng cách bão hòa tần suất từ và chuẩn hóa độ dài văn bản. Tuy nhiên, nhược điểm về phụ thuộc từ khóa khiến BM25 không phù hợp khi đối mặt với sự đa dạng ngôn ngữ tiếng Việt.

Để làm rõ cách triển khai Baseline 1 trong thực tế, đoạn mã sau minh họa toàn bộ quy trình TF-IDF: từ tách từ ViTokenizer, vector hóa sparse, tính Cosine Similarity đến đánh giá $P@K$ trên tập test.

```

# File: train_phobert_faq.py - Phân xây dựng và đánh giá TF-IDF Baseline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from pyvi import ViTokenizer
import numpy as np

# Bước 1: Tách từ tiếng Việt - bắt buộc để nhất quán với PhoBERT
def word_segment(text: str) -> str:
    return ViTokenizer.tokenize(text)

faq_questions_seg = [word_segment(q) for q in faq_questions]

# Bước 2: Xây dựng TF-IDF Vectorizer (sparse vector, không cần GPU)
tfidf = TfidfVectorizer(analyzer="word")
tfidf_matrix = tfidf.fit_transform(faq_questions_seg)
# tfidf_matrix: shape (n_faq, vocab_size) - sparse matrix

# Bước 3: Truy xuất FAQ bằng Cosine Similarity
def retrieve_tfidf(query: str, top_k: int = 5):
    q_seg = word_segment(query)
    q_vec = tfidf.transform([q_seg])
    scores = cosine_similarity(q_vec, tfidf_matrix).flatten()
    top_idx = np.argsort(scores)[: -1][:top_k]
    return [(faq_ids[i], scores[i]) for i in top_idx]

# Bước 4: Đánh giá P@K trên tập test
hits = {1: 0, 3: 0, 5: 0}
for query, gold_id in test_pairs:
    retrieved = [r[0] for r in retrieve_tfidf(query, top_k=5)]
    for k in [1, 3, 5]:
        if gold_id in retrieved[:k]: hits[k] += 1
for k in [1, 3, 5]:
    print(f"TF-IDF P@{k} = {hits[k]/len(test_pairs):.4f}")

```

Hình 3.1: Code minh họa - Xây dựng và đánh giá TF-IDF Baseline

3.1.2. Baseline 2: Generative QA (Dùng trực tiếp LLM)

Phương pháp thứ hai là sử dụng trực tiếp mô hình ngôn ngữ lớn để trả lời câu hỏi mà không thông qua bước retrieval.

Ưu điểm của phương pháp này là khả năng xử lý các câu hỏi có tính suy luận, câu hỏi mở và câu hỏi không có trong FAQ. Tuy nhiên, nhược điểm nghiêm trọng là vấn đề hallucination: LLM có xu hướng sinh ra câu trả lời nghe có vẻ hợp lý nhưng không chính xác về mặt thực tế, đặc biệt với các thông tin chi tiết về quy định nội bộ của trường. Ngoài ra, phương pháp này đòi hỏi tài nguyên tính toán cao và không thể đảm bảo tính nhất quán với nội dung FAQ chính thức.

3.1.3. Baseline 3: Thuần RAG (Retrieval-Augmented Generation)

Phương pháp RAG kết hợp ưu điểm của retrieval và generative: bước retrieval tìm top-k câu hỏi-trả lời liên quan nhất từ FAQ, sau đó đưa chúng vào context cho LLM để sinh câu trả lời hoàn chỉnh.

Ưu điểm của RAG thuần túy là câu trả lời được sinh ra có cơ sở từ nội dung FAQ thực tế, giảm thiểu hallucination. LLM có thể tổng hợp thông tin từ nhiều mục FAQ, diễn đạt lại theo ngữ cảnh câu hỏi cụ thể, và trả lời được các câu hỏi liên quan.

Tuy nhiên, RAG thuần túy vẫn có nhược điểm: nếu bước retrieval trả về các câu FAQ không liên quan (do câu hỏi ngoài phạm vi), LLM vẫn sẽ cố gắng sinh câu trả lời từ context sai lệch; chi phí tính toán cao hơn vì luôn phải gọi LLM cho mọi câu hỏi; với các câu hỏi đơn giản đã có câu trả lời trực tiếp trong FAQ, việc sinh lại câu trả lời là dư thừa và có thể làm mất thông tin.

Để cụ thể hóa cơ chế RAG thuần túy (Baseline 3), đoạn mã dưới đây minh họa hàm `llm_generate_answer()` - xây dựng context từ top-K FAQ và gửi đến Ollama API để sinh câu trả lời tổng hợp.

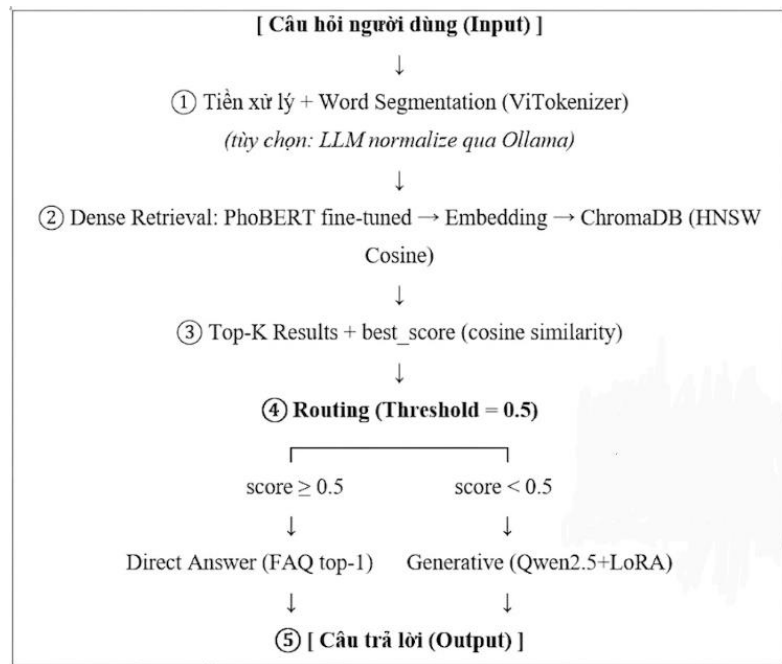
```
# File: demo.py — Hàm xây dựng Prompt RAG và gọi Ollama API (Mode 2)
RAG_PROMPT = """"Bạn là trợ lý tư vấn học vụ HCMUTE.
Dựa trên các FAQ dưới đây, hãy trả lời câu hỏi bằng tiếng Việt.
=== THÔNG TIN FAQ ===
{context}
=== CÂU HỎI ===
{question}"""""

def llm_generate_answer(question: str, metadatas: list) -> str:
    # Bước 1: Ghép top-K FAQ thành context
    parts = [f[FAQ {i}] Q: {m["question"]} A: {m["answer"]}']
    for i, m in enumerate(metadatas, 1)]
    context = "\n\n".join(parts)
    # Bước 2: Gọi Ollama API với prompt RAG đã xây dựng
    prompt = RAG_PROMPT.format(context=context, question=question)
    resp = requests.post(OLLAMA_URL, json={
        "model": OLLAMA_MODEL, "prompt": prompt, "stream": False,
        "options": {"temperature": 0.3, "num_predict": 512},
    }, timeout=120)
    return resp.json().get("response", "").strip()
```

Hình 3.2: Code minh họa - Hàm `llm_generate_answer()` (RAG với Ollama API)

3.2. Kiến trúc đề xuất: Hybrid FAQ + Generative RAG

Kiến trúc đề xuất kết hợp ưu điểm của cả ba phương pháp baseline và bổ sung cơ chế phân luồng thông minh dựa trên ngưỡng độ tương đồng.



Hình 3.3: Kiến trúc tổng quan hệ thống Hybrid FAQ + Generative RAG

3.2.1. Luồng xử lý giao tiếp (Routing Mechanism – Ngưỡng Threshold)

Cơ chế phân luồng là thành phần phân biệt kiến trúc đề xuất với các baseline. Sau khi thực hiện retrieval và tính độ tương đồng cosine giữa embedding câu hỏi người dùng và embedding câu hỏi FAQ, hệ thống sử dụng `best_score` (độ tương đồng của kết quả top-1) để quyết định luồng xử lý tiếp theo.

Ngưỡng THRESHOLD được đặt ở giá trị 0.6 trên thang điểm cosine similarity $[0, 1]$. Giá trị này được xác định qua thực nghiệm: một câu hỏi có điểm cosine similarity ≥ 0.6 với câu hỏi FAQ top-1 được coi là trong phạm vi và có thể trả lời trực tiếp từ FAQ; ngược lại, điểm < 0.6 gợi ý câu hỏi không có tương đồng đủ mạnh trong kho FAQ, có thể là câu hỏi ngoài phạm vi hoặc câu hỏi cần suy luận thêm.

Bốn chế độ hoạt động được định nghĩa như sau: Mode 1 (Only Retrieval) trả lời trực tiếp từ FAQ nếu $\text{score} \geq \text{threshold}$, ngược lại trả về thông báo "không thể trả lời". Mode 2 (Pure RAG) luôn gọi LLM Ollama để sinh câu trả lời từ top-k FAQ bất kể ngưỡng. Mode 3 (Hybrid) kết hợp: trả lời trực tiếp từ FAQ khi $\text{score} \geq \text{threshold}$ và fallback sang mô hình sinh LoRA khi $\text{score} < \text{threshold}$. Mode 4 (Pure Generative) bỏ qua hoàn toàn retrieval, chỉ dùng Qwen2.5+LoRA để sinh câu trả lời.

Input: question (câu hỏi đã chuẩn hóa), mode=3
Output: answer (câu trả lời cuối cùng), out_of_scope (bool)
function hybrid_routing(question, mode=3):

1. $q_emb \leftarrow \text{PhoBERT_finetune.encode}(\text{question})$ // Dense embedding
2. $\text{results} \leftarrow \text{ChromaDB.query}(q_emb, \text{top_k}=5)$ // HNSW cosine search
3. $\text{best_score} \leftarrow 1 - \text{results.distances}[0]$ // Cosine similarity
4. $\text{top1_meta} \leftarrow \text{results.metadataas}[0]$
5. IF $\text{mode} == 1$:
IF $\text{best_score} \geq \text{THRESHOLD} (0.6)$:
RETURN top1_meta.answer , $\text{out_of_scope}=\text{False}$
ELSE:
RETURN CANNOT_ANSWER_MSG , $\text{out_of_scope}=\text{True}$
6. IF $\text{mode} == 2$:
$\text{context} \leftarrow \text{format_top_k}(\text{results.metadataas})$
RETURN $\text{Ollama_RAG}(\text{question}, \text{context})$, $\text{out_of_scope}=\text{False}$
7. IF $\text{mode} == 3$ (PROPOSED HYBRID):
IF $\text{best_score} \geq \text{THRESHOLD}$:
RETURN top1_meta.answer , $\text{out_of_scope}=\text{False}$ // Direct FAQ
ELSE:
RETURN $\text{Qwen_LoRA_generate}(\text{question})$, ... // Generative fallback
8. IF $\text{mode} == 4$:
RETURN $\text{Qwen_LoRA_generate}(\text{question})$, $\text{out_of_scope}=\text{False}$

Thuật toán 3.1: Cơ chế phân luồng Hybrid Routing

3.2.2. Pipeline truy xuất – sinh hoàn chỉnh

Pipeline hoàn chỉnh của hệ thống bao gồm năm giai đoạn chính, liên kết chặt chẽ với nhau từ khi nhận câu hỏi đến khi trả về câu trả lời cuối cùng.

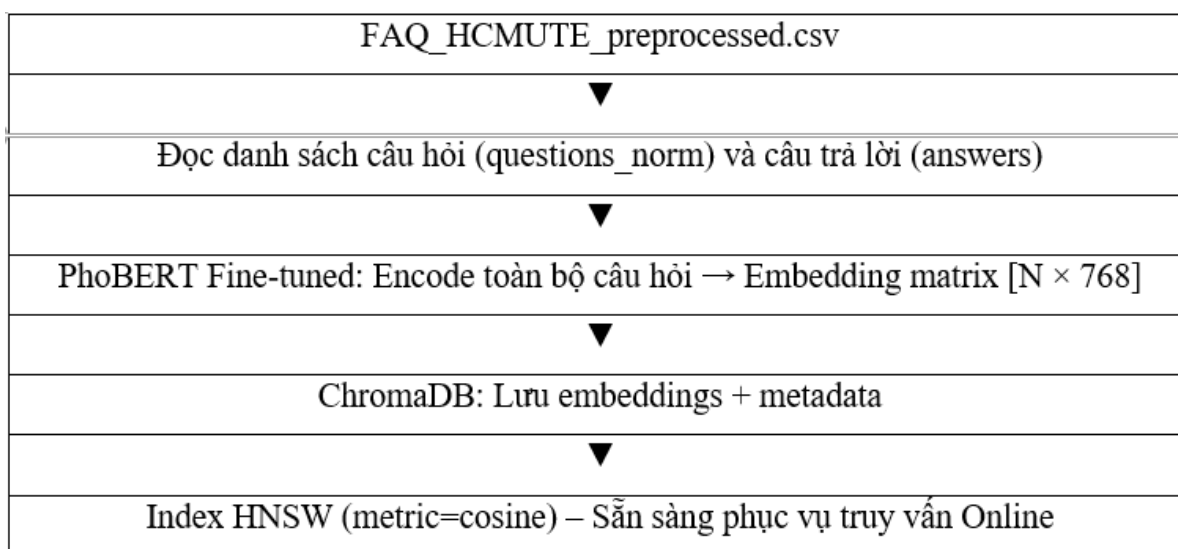
Giai đoạn 1 (Tiền xử lý câu hỏi): Câu hỏi thô từ người dùng trải qua pipeline preprocessing (mô tả phần 2.3). Hệ thống có thể gọi LLM (Ollama) để chuẩn hóa ngữ nghĩa câu hỏi trước (sửa lỗi chính tả, bỏ từ đệm "cho em hỏi", "dạ",...). Cuối cùng, câu hỏi được tách từ bằng ViTokenizer để tạo input phù hợp cho PhoBERT.

Giai đoạn 2 (Dense Embedding): Câu hỏi đã tiền xử lý được đưa qua mô hình PhoBERT fine-tuned để tạo vector embedding chiều cao, được chuẩn hóa L2. Embedding vector biểu diễn ngữ nghĩa của câu hỏi trong không gian vector liên tục.

Giai đoạn 3 (Vector Search trên ChromaDB): Vector embedding câu hỏi được dùng để truy vấn ChromaDB PersistentClient, tìm kiếm top-k câu hỏi FAQ có độ tương đồng cosine cao nhất. ChromaDB sử dụng cấu trúc HNSW (Hierarchical Navigable Small World) với metric cosine, đảm bảo tốc độ tìm kiếm gần đúng trên tập FAQ lớn.

Giai đoạn 4 (Routing và phân luồng): best_score được tính từ khoảng cách cosine của kết quả top-1 (best_score = 1 - distance[0]). Cơ chế routing theo Thuật toán 3.1 quyết định luồng xử lý tiếp theo dựa trên mode và threshold.

Giai đoạn 5 (Sinh câu trả lời và trả về): Tùy theo luồng được chọn, hệ thống trả về câu trả lời trực tiếp từ metadata FAQ (mode 1/3 khi score đủ cao) hoặc sinh câu trả lời thông qua LLM (mode 2: Ollama RAG; mode 3 fallback: Qwen2.5+LoRA local; mode 4: pure generative).



Hình 3.4: Pipeline xây dựng index vector (offline)

Hai hàm build_index() và search_vector() triển khai trực tiếp Giai đoạn 2 và 3 của pipeline: embedding hàng loạt FAQ và truy vấn ChromaDB HNSW tại thời điểm suy diễn.

```
# File: demo.py — build_index() và search_vector() với ChromaDB
import chromadb

def build_index(encoder, client: chromadb.ClientAPI):
    """Offline: Nhúng toàn bộ FAQ vào ChromaDB (HNSW cosine)."""
    df = pd.read_csv(DATA_PATH, encoding="utf-8").dropna()
    collection = client.get_or_create_collection(
        "hcmute_faq", metadata={"hnsw:space": "cosine"})
    # Nhúng hàng loạt; L2-normalize → cos sim = dot product
    embs = encoder.encode(df["question"].tolist(), batch_size=64).tolist()
    collection.add(
        ids=[str(r.faq_id) for r in df.itertuples()],
        embeddings=embs,
        metadatas=[{"question": r.question, "answer": r.answer,
                     "faq_id": str(r.faq_id)} for r in df.itertuples()],
    )
    return collection, len(df)

def search_vector(collection, encoder, n_faq, query, top_k=5):
    """Online: Nhúng câu hỏi truy vấn → tìm top_k FAQ gần nhất."""
    q_emb = encoder.encode([query]).tolist()
    results = collection.query(
        query_embeddings=q_emb,
        n_results=min(top_k, n_faq),
        include=["distances", "metadatas"],
    )
    # ChromaDB: distances[i] = 1 - cos_sim
    # → best_score = 1 - results["distances"][0][0]
    return results
```

Hình 3.5: Code minh họa - build_index() và search_vector() với ChromaDB

3.3. Huấn luyện và tinh chỉnh mô hình (Fine-tuning)

3.3.1. Fine-tune PhoBERT cho FAQ Retrieval bằng tập dữ liệu Paraphrase

Mô hình PhoBERT phiên bản vinai/phobert-base-v2 được sử dụng làm nền tảng cho encoder biểu diễn câu hỏi. PhoBERT được phát triển bởi VinAI Research, được huấn luyện trước trên corpus tiếng Việt quy mô lớn, đã chứng minh hiệu quả vượt trội so với các mô hình đa ngôn ngữ trên các tác vụ NLP tiếng Việt.

PhoBERT được tích hợp vào framework SentenceTransformer để tạo ra sentence embedding – vector biểu diễn toàn bộ câu trong một không gian ngữ nghĩa liên tục. Quá trình fine-tune được thực hiện theo mô hình Metric Learning sử dụng tập dữ liệu paraphrase đã xây dựng ở Chương 2.

Chiến lược chia dữ liệu theo nhóm đảm bảo không có data leakage: mỗi faq_id chỉ xuất hiện trong đúng một tập. Cụ thể, N FAQ được shuffle với random seed, sau đó

chia theo tỷ lệ 80/10/10. Tất cả các cặp paraphrase của một `faq_id` được giữ cùng nhau trong cùng một tập, đảm bảo tập test chứa các FAQ hoàn toàn mới.

```
# File: train_phobert_faq.py — Chia dữ liệu theo nhóm FAQ (Group-based Split)
from sklearn.model_selection import GroupShuffleSplit

df = pd.read_csv("FAQ_HCMUTE_paraphrases_10502.csv")
groups = df["faq_id"].values # nhãn nhóm

# Bước 1: Tách test (10%) — mỗi faq_id chỉ ở một tập
gss = GroupShuffleSplit(n_splits=1, test_size=0.10, random_state=42)
trainval_idx, test_idx = next(gss.split(df, groups=groups))

# Bước 2: Tách val (≈10% tổng) từ phần còn lại
df_tv = df.iloc[trainval_idx]
g_tv = groups[trainval_idx]
gss2 = GroupShuffleSplit(n_splits=1, test_size=0.111, random_state=42)
train_idx, val_idx = next(gss2.split(df_tv, groups=g_tv))

df_train, df_val, df_test = df_tv.iloc[train_idx], df_tv.iloc[val_idx], df.iloc[test_idx]
# Kết quả: Train ~8401 | Val ~951 | Test ~1150

# Kiểm tra: không có faq_id nào xuất hiện ở nhiều tập
assert set(df_train.faq_id) & set(df_test.faq_id) == set()
assert set(df_val.faq_id) & set(df_test.faq_id) == set()
```

Hình 3.6: Code minh họa - Chia dữ liệu theo nhóm `faq_id` (`GroupShuffleSplit`)

Lớp `FinetunedEncoder` đóng gói `PhoBERT` fine-tuned cùng bước tách từ `ViTokenizer`. Nhất quán word segmentation giữa train và inference là điều kiện bắt buộc để embedding câu hỏi và FAQ nằm trong cùng không gian vector:

```

# File: demo.py — Lớp FinetunedEncoder (PhoBERT fine-tuned + ViTokenizer)
from sentence_transformers import SentenceTransformer
from pyvi import ViTokenizer
import numpy as np

class FinetunedEncoder:
    """Wrapper tích hợp word segmentation vào pipeline encode."""

    def __init__(self, model_path: str, device: str = "cpu"):
        self.model = SentenceTransformer(model_path, device=device)

    def _segment(self, text: str) -> str:
        """Tách từ tiếng Việt — bắt buộc nhất quán train và inference."""
        return ViTokenizer.tokenize(text)

    def encode(self, texts, batch_size: int = 32,
               normalize: bool = True) -> np.ndarray:
        if isinstance(texts, str): texts = [texts]
        segmented = [self._segment(t) for t in texts]
        embeddings = self.model.encode(
            segmented, batch_size=batch_size,
            normalize_embeddings=normalize, # L2-norm → cosine = dot
            show_progress_bar=False)
        return embeddings # shape: (len(texts), 768)

# Ví dụ: encoder.encode("Hiệu trưởng là ai") → (768,)

```

Hình 3.7: Code minh họa - Lớp FinetunedEncoder (PhoBERT + ViTokenizer)

Bảng dưới đây là minh họa thuật toán lúc fine-tune:

Input: train_examples [(s1, s2)] – positive pairs
val_examples – tập kiểm định
Output: model fine-tuned → ./phobert_faq_retrieval
function finetune_phobert(train_examples, val_examples):
1. model ← SentenceTransformer("vinai/phobert-base-v2")
2. // Áp dụng word segmentation cho tất cả câu

$\text{train_seg} \leftarrow [\text{word_segment}(s1, s2) \text{ for } (s1, s2) \text{ in train_examples}]$
3. $\text{loss} \leftarrow \text{MultipleNegativesRankingLoss}(\text{model})$
// Với mỗi batch, negative = các s2 khác trong batch
4. $\text{evaluator} \leftarrow \text{BinaryClassificationEvaluator}(\text{val_examples})$
5. $\text{trainer} \leftarrow \text{SentenceTransformerTrainer}(\text{model}=\text{model}, \text{loss}=\text{loss}, \text{args}=\text{TrainingArgs}(\text{epochs}=2, \text{batch_size}=32, \text{lr}=1\text{e-}5, \text{eval_steps}=\text{auto}), \text{evaluator}=\text{evaluator})$
6. $\text{trainer.train}()$
7. $\text{model.save}("./\text{phobert_faq_retrieval}")$

Thuật toán 3.2: Fine-tune PhoBERT cho FAQ Retrieval

3.3.2. Cấu hình Loss, Optimizer và hàm đánh giá

Hàm mất mát MultipleNegativesRankingLoss (MNRL) được chọn làm hàm loss chính cho quá trình fine-tune. MNRL là hàm loss phù hợp nhất cho bài toán retrieval với dữ liệu positive pairs: với mỗi batch có B cặp $(s1_i, s2_i)$, mỗi cặp positive $(s1_i, s2_i)$ được học bằng cách tối đa hóa độ tương đồng của chúng, đồng thời tối thiểu hóa độ tương đồng với tất cả các $s2_j$ ($j \neq i$) trong cùng batch. Hiệu quả là mô hình học được $(2B - 2)$ cặp negative từ một batch mà không cần nhãn negative rõ ràng, cho phép training hiệu quả với tập dữ liệu paraphrase chỉ chứa positive pairs.

```
# Công thức MultipleNegativesRankingLoss (MNRL)
# Với batch B cặp: {(q_1, a_1), ..., (q_B, a_B)}
# Q, A: ma trận embedding L2-normalized, shape (B, d)
# Ma trận similarity: S = Q @ A.T → S[i][j] = cos(q_i, a_j)
# S[i][i] = positive; S[i][j], j≠i = in-batch negatives
#
#           exp(S[i][i])
# L_i = -log -----
#           Σ_{j=1}^{B} exp(S[i][j])
#
# L = (1/B) * Σ L_i
#
# Mỗi batch B cặp → B*(B-1) negative pairs tự động
# → Không cần nhãn negative rõ ràng, phù hợp dữ liệu paraphrase
#
# Triển khai với sentence-transformers:
from sentence_transformers import losses
loss = losses.MultipleNegativesRankingLoss(model)
# Sử dụng: loss([(s1_i, s2_i)] list of positive pairs) → scalar
```

Hình 3.8: Công thức toán học MultipleNegativesRankingLoss và cách triển khai

Về tham số tối ưu, optimizer AdamW được sử dụng với learning rate $1e-5$. Batch size được đặt ở 32 khi chạy trên GPU CUDA. Số epoch huấn luyện là 2 – đủ để mô hình hội tụ mà không bị overfitting do tập dữ liệu paraphrase tương đối nhỏ.

Quá trình huấn luyện được theo dõi qua callback LossHistoryCallback tùy chỉnh, thu thập train loss và eval loss theo từng bước để vẽ biểu đồ hội tụ sau huấn luyện. Đánh giá định kỳ trên tập validation được thực hiện thông qua BinaryClassificationEvaluator của SentenceTransformers, đo F1 và AUC trên các cặp positive/negative nhị phân.

Các hàm đánh giá được triển khai bao gồm: Cosine Similarity trung bình trên tập test (đo mức độ kéo gần cặp positive); Precision@K ($P@1$, $P@3$, $P@5$) đo tỷ lệ câu hỏi gốc xuất hiện trong top-k kết quả retrieval khi dùng paraphrase làm truy vấn; và chỉ số margin ($\text{sim}(\text{positive}) - \text{sim}(\text{negative})$) đo khả năng phân biệt giữa cặp tương đồng và cặp không tương đồng. Cả ba mô hình được so sánh: TF-IDF baseline, PhoBERT gốc (chưa fine-tune), và PhoBERT fine-tuned.

3.3.3. Fine-tune mô hình sinh (Qwen 2.5 7B)

Mô hình sinh ngôn ngữ Qwen2.5-7B-Instruct được fine-tune để đảm nhận vai trò generative component trong kiến trúc hybrid. Qwen2.5-7B-Instruct được chọn vì có hiệu năng cao trên tiếng Việt, hỗ trợ template chat instruction-following, và kích thước 7B vừa đủ để chạy trên GPU T4 16GB với quantization 4-bit.

Kỹ thuật LoRA (Low-Rank Adaptation) được áp dụng để giảm thiểu số lượng tham số cần huấn luyện. LoRA chèn các ma trận rank thấp vào các lớp attention của transformer mà không cần cập nhật toàn bộ trọng số mô hình. Điều này cho phép fine-tune hiệu quả trên phần cứng hạn chế trong khi vẫn đạt khả năng thích ứng.

Dữ liệu fine-tune cho Qwen được xây dựng từ tập FAQ_HCMUTE theo định dạng instruction-following: mỗi mẫu là một cặp (instruction: câu hỏi về HCMUTE, output: câu trả lời chính thức). Quá trình fine-tune được thực hiện bằng Unsloth framework để tăng tốc và giảm VRAM, sử dụng định dạng 4-bit quantization (NF4) với base model unsloth/qwen2.5-7b-instruct-bnb-4bit.

Input: FAQ_HCMUTE dataset (câu hỏi → câu trả lời)
Output: LoRA adapter → ./qwen25_7b_instruct_lora
function finetune_qwen_lora(faq_dataset):
1. base_model ← load_4bit("unsloth/qwen2.5-7b-instruct-bnb-4bit")

2. model ← FastLanguageModel.get_peft_model(
base_model,
r=16, // LoRA rank
lora_alpha=16, // Scaling factor
target_modules=["q_proj","k_proj","v_proj","o_proj",
"gate_proj","up_proj","down_proj"],
lora_dropout=0,
use_gradient_checkpointing=True
)
3. dataset ← format_instruction(faq_dataset)
// Template: < im_start >user\nCâu hỏi\n< im_end >
// < im_start >assistant\nCâu trả lời< im_end >
4. trainer ← SFTTrainer(
model, dataset,
max_seq_length=2048,
args=TrainingArgs(epochs=2, batch_size=2, lr=2e-4,
warmup_ratio=0.03, lr_scheduler=cosine)
)
5. trainer.train()
6. model.save_pretrained("./qwen25_7b_instruct_lora")
7. // Inference: load base 4bit + merge LoRA adapter
base ← AutoModelForCausalLM(base, load_in_4bit=True)
model ← PeftModel.from_pretrained(base, lora_dir).eval()

Thuật toán 3.3: Fine-tune Qwen2.5-7B với LoRA

Kỹ thuật LoRA thêm hai ma trận rank thấp B và A vào các lớp attention mà không cập nhật W_0 . Công thức toán học và cấu hình thực tế được minh họa dưới đây:

```

# Công thức LoRA (Low-Rank Adaptation)
# Trọng số gốc (frozen):  $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$ 
# LoRA thêm:  $\Delta W = B \cdot A$ 
#  $B \in \mathbb{R}^{d_{out} \times r}$ ,  $A \in \mathbb{R}^{r \times d_{in}}$ ,  $r \ll d$ 
#
# Forward pass:  $h = W_0 \cdot x + (\alpha/r) \cdot B \cdot A \cdot x$ 
#  $r = 16$ ,  $\alpha = 16 \rightarrow$  scaling factor  $\alpha/r = 1.0$ 
# A khởi tạo Gaussian; B khởi tạo 0  $\rightarrow \Delta W = 0$  khi bắt đầu
#
# Số tham số LoRA (Qwen2.5-7B,  $d=4096$ , 7 modules):
# Mỗi layer:  $2 \times (4096 \times 16) = 131,072$ 
# Tổng  $\approx 37M$  params  $\approx 0.53\%$  tổng – giảm  $\sim 95\%$  so với full fine-tune

# Cấu hình PEFT trong finetune_mauV2.ipynb:
model = FastLanguageModel.get_peft_model(
    base_model,
    r=16, lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                  "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0,
    use_gradient_checkpointing=True,
)

```

Hình 3.9: Công thức toán học LoRA và cấu hình thực tế ($r=16$, $\alpha=16$, 0.53% tham số)

Mô hình Qwen2.5+LoRA được load bằng cách kết hợp base model 4-bit và LoRA adapter. Câu hỏi người dùng được format theo chat template của Qwen2.5 trước khi đưa vào mô hình sinh.

CHƯƠNG 4: THỰC NGHIỆM, KẾT QUẢ VÀ SO SÁNH

4.1. Môi trường và thiết lập thực nghiệm

Toàn bộ quá trình huấn luyện, đánh giá và triển khai hệ thống được thực hiện trên nền tảng đám mây Lightning AI, sử dụng GPU NVIDIA Tesla T4. Lựa chọn phần cứng này đảm bảo đủ tài nguyên cho tác vụ: fine-tune mô hình PhoBERT. Trên Google colab fine-tune mô hình sinh Qwen2.5-7B-Instruct với quantization 4-bit.

Thành phần	Chi tiết
Nền tảng	Lightning AI (cloud GPU)
GPU	NVIDIA Tesla T4 – 16 GB VRAM
CPU / RAM	8 vCPU, 32 GB RAM
Hệ điều hành	Ubuntu 22.04 LTS
Python	3.10
PyTorch	2.2.0 + CUDA 12.1
Transformers	4.40.x (HuggingFace)
Sentence-Transformers	3.x
ChromaDB	0.5.x (PersistentClient, HNSW cosine)
Mô hình retrieval base	vinai/phobert-base-v2
Mô hình sinh base	unsloth/qwen2.5-7b-instruct-bnb-4bit (NF4)
LLM sinh paraphrase	Qwen3.5:9b qua Ollama (localhost)
Tập dữ liệu huấn luyện	FAQ_HCMUTE_paraphrases_10502.csv / .jsonl
Cấu hình fine-tune PhoBERT	batch=32, lr=1e-5, epochs=2, loss=MultipleNegativesRankingLoss
Cấu hình fine-tune Qwen LoRA	r=16, lora_alpha=16, batch=2, lr=2e-4, epochs=2, scheduler=cosine

Bảng 4.1: Cấu hình môi trường và thiết lập thực nghiệm

Về dữ liệu thực nghiệm, tập test được xây dựng theo chia nhóm FAQ: các cặp thuộc cùng một faq_id chỉ xuất hiện trong một tập duy nhất. Tập test cuối cùng gồm 1.150 cặp truy vấn, tương ứng với khoảng 10% tổng số cặp, và bao phủ các FAQ hoàn toàn không được nhìn thấy trong quá trình huấn luyện. Cách chia này đảm bảo đánh giá phản ánh đúng khả năng tổng quát hóa của mô hình trong điều kiện thực tế.

4.2. Các tiêu chí đánh giá

4.2.1. Đánh giá truy xuất: Precision@k (P@1, P@3, P@5)

Bài toán FAQ retrieval được đánh giá theo chuẩn Information Retrieval (IR) với tiêu chí chính là Precision@K (viết tắt P@K). Đây là chỉ số đo tỷ lệ truy vấn mà câu hỏi FAQ đúng xuất hiện trong K kết quả được trả về hàng đầu (top-K). Với bài toán này, mỗi truy vấn là một câu paraphrase, và câu hỏi FAQ đúng (ground truth) là câu hỏi gốc mà paraphrase được sinh ra từ đó.

$$P@K = \frac{\text{Số truy vấn có câu FAQ đúng nằm trong top-K kết quả}}{\text{Tổng số truy vấn}}$$

Trong đó: $K \in \{1, 3, 5\}$ | $P@K \in [0, 1]$ | $P@K_1 \leq P@K_2$ nếu $K_1 \leq K_2$

Công thức 4.1: Precision@K

Ba mức K được lựa chọn để đánh giá toàn diện: P@1 là tiêu chí nghiêm ngặt nhất, đo khả năng trả về đúng kết quả ngay ở vị trí đầu tiên. P@3 và P@5 đánh giá khả năng recall trong trường hợp hệ thống cung cấp nhiều gợi ý (RAG top-K).

Quy trình đánh giá được thực hiện như sau: với mỗi câu paraphrase trong tập test, mô hình mã hóa câu paraphrase thành embedding vector, sau đó so sánh với toàn bộ embedding của các câu hỏi FAQ trong kho tri thức. Top-K câu FAQ có độ tương đồng cosine cao nhất được trả về. Kết quả được coi là đúng (hit) nếu câu hỏi FAQ gốc xuất hiện trong top-K. Tỷ lệ hit trên toàn tập test chính là P@K.

Để so sánh đầy đủ, ba mô hình được đánh giá với cùng tập test và cùng quy trình: TF-IDF (baseline truyền thống), PhoBERT gốc chưa fine-tune (vinai/phobert-base-v2), và PhoBERT sau fine-tune với tập paraphrase HCMUTE. Đánh giá TF-IDF sử dụng TfidfVectorizer của scikit-learn với analyzer='word', trên tập câu hỏi FAQ đã qua word segmentation ViTokenizer.

4.2.2. Cosine Similarity và Margin Analysis

Bên cạnh P@K, hai tiêu chí bổ sung được sử dụng để phân tích sâu chất lượng biểu diễn ngữ nghĩa của mô hình: Cosine Similarity trung bình và Similarity Margin.

Cosine Similarity trung bình (Avg Cosine Sim) được tính là giá trị trung bình của độ tương đồng cosine giữa embedding của câu paraphrase truy vấn và embedding của câu FAQ đúng (positive pair) trên toàn tập test. Chỉ số này đo mức độ "kéo gần" mà mô hình đạt được giữa các câu có cùng ý nghĩa.

$$\text{Avg_cos_sim} = \frac{1}{N} \sum_{i=1}^N \cos(\text{embed}(\text{query}_i), \text{embed}(\text{faq_correct}_i))$$

$$\cos(u, v) = (u \cdot v) / (\|u\| \times \|v\|) \quad [\text{L2-normalized vectors: } \cos(u, v) = u \cdot v]$$

Công thức 4.2: Cosine Similarity trung bình trên positive pairs

Similarity Margin là chỉ số quan trọng hơn để đánh giá chất lượng ranking. Margin được định nghĩa là hiệu số giữa độ tương đồng trung bình với FAQ đúng (sim_positive) và độ tương đồng trung bình với các FAQ sai (sim_negative):

$$\begin{aligned} \text{Margin} &= \text{sim_positive} - \text{sim_negative} \\ \text{sim_positive} &= \text{Avg}[\cos(\text{query}, \text{FAQ_đúng})] \\ \text{sim_negative} &= \text{Avg}[\cos(\text{query}, \text{FAQ_sai})] \end{aligned}$$

Công thức 4.3: Similarity Margin

Margin cao đồng nghĩa với mô hình có khả năng phân biệt rõ ràng giữa câu hỏi liên quan và không liên quan trong không gian embedding. Đây là điều kiện cốt lõi để hệ thống retrieval hoạt động tốt. Một mô hình có thể có Avg Cosine Sim cao nhưng Margin thấp (nếu cả FAQ đúng lẫn FAQ sai đều có similarity cao), kết quả là ranking kém dù mô hình hiểu câu hỏi về mặt ngữ nghĩa tổng thể.

Chính sự tách biệt giữa hai chỉ số này giải thích một hiện tượng thú vị trong kết quả thực nghiệm: mặc dù PhoBERT gốc có Avg Cosine Sim cao hơn PhoBERT fine-tuned (0.8835 so với 0.8193), nhưng PhoBERT fine-tuned lại cho P@K cao hơn rất nhiều do Margin tốt hơn đáng kể. Phân tích chi tiết được trình bày trong mục 4.3.1.

4.3. Kết quả đánh giá và so sánh phương pháp

4.3.1. So sánh TF-IDF, PhoBERT gốc và PhoBERT Fine-tuned

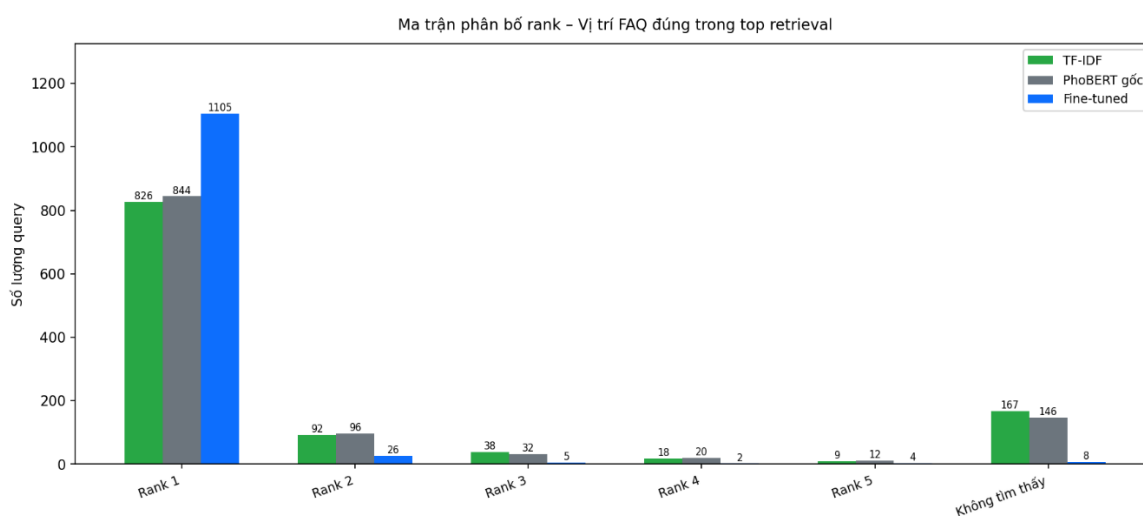
Bảng 4.2 trình bày kết quả đánh giá tổng hợp của ba phương pháp trên tập test gồm 1.150 câu paraphrase. Kết quả cho thấy PhoBERT fine-tuned vượt trội rõ rệt so với cả TF-IDF và PhoBERT gốc trên tất cả các tiêu chí P@K.

Tiêu chí đánh giá	TF-IDF	PhoBERT gốc	PhoBERT Fine-tuned	$\Delta(\text{FT vs. Gốc})$
Cosine Similarity	—	0.8835	0.8193	−0.0642
P@1	0.7183	0.7339	0.9609 ★	+0.2270

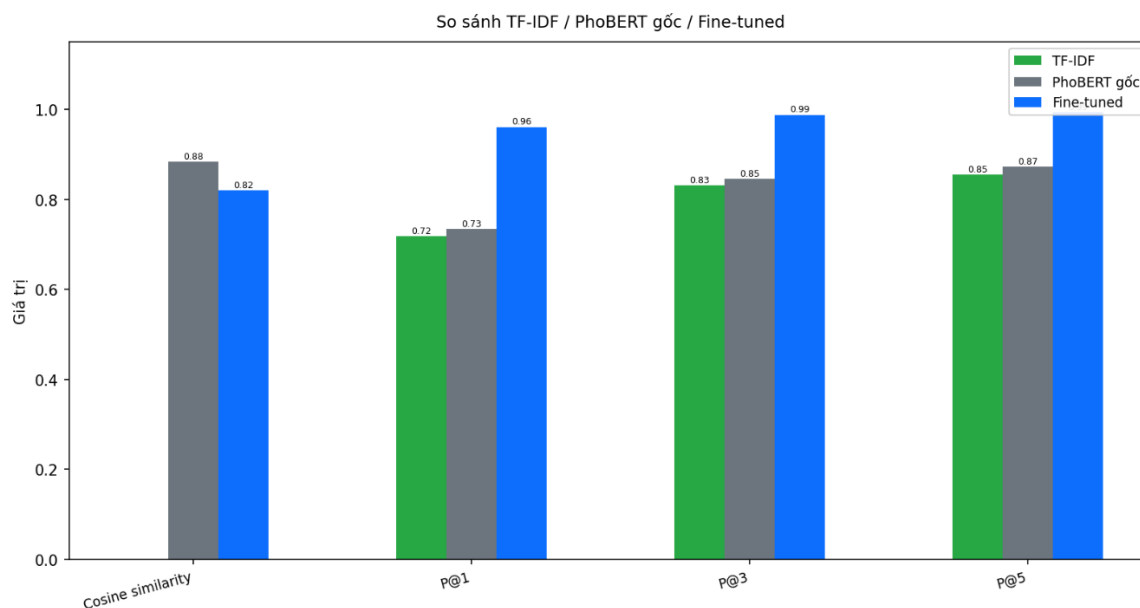
P@3	0.8313	0.8452	0.9878 ★	+0.1426
P@5	0.8548	0.8730	0.9930 ★	+0.1200
sim_positive	–	0.8835	0.8193	–0.0642
sim_negative	–	0.6955	0.0626	–0.6329
Margin (pos – neg)	–	0.1881	0.7567 ★	+0.5686
Số truy vấn	1.150	1.150	1.150	–
Không tìm thấy	167 (14,5%)	146 (12,7%)	8 (0,7%) ★	–138 truy vấn

Bảng 4.2: Kết quả so sánh TF-IDF, PhoBERT gốc và PhoBERT Fine-tuned

Phân tích chi tiết theo phân bố rank trong Hình 4.1. Phân bố này cho thấy sự cải thiện vượt bậc: PhoBERT fine-tuned xếp đúng FAQ ở Rank 1 cho 1.105/1.150 truy vấn (96,09%), so với 844 của PhoBERT gốc (73,39%) và 826 của TF-IDF (71,83%). Đặc biệt, số truy vấn hoàn toàn không tìm thấy giảm từ 167 (TF-IDF), 146 (PhoBERT gốc) xuống còn chỉ 8 (PhoBERT fine-tuned) – giảm hơn 20 lần so với TF-IDF.



Hình 4.1: Ma trận phân bố rank – Số lượng truy vấn theo vị trí FAQ đúng trong kết quả top-5



Hình 4.2: Biểu đồ so sánh các tiêu chí đánh giá giữa ba phương pháp

Kết quả xác nhận giả thuyết ban đầu: fine-tune PhoBERT với tập paraphrase HCMUTE làm cải thiện đáng kể khả năng truy xuất ngữ nghĩa trong domain tiếng Việt chuyên biệt. Cụ thể, ba nhận xét chính được rút ra từ dữ liệu thực nghiệm:

TF-IDF vs PhoBERT gốc: Hai phương pháp cho kết quả tương đương nhau (P@1: 0.7183 vs. 0.7339, chênh lệch chỉ 1.56%). Nhưng đừng vội kết luận rằng PhoBERT gốc kém hơn, chúng ta hãy xét kỹ hơn vào một câu cụ thể bởi vì data paraphrase có nhiều câu chỉ khác vài từ.

```

"query": "những gì cần chuẩn bị cho buổi học của sinh viên?",
"ground_truth": {
  "question": "sinh viên nên chuẩn bị gì trước khi đến lớp?",
  "answer": "Đọc kỹ đề cương học phần của môn học."
},
"correct_rank": 2,
"top_k": [
  {
    "rank": 1,
    "question": "những việc cần làm sau khi kết thúc buổi học là gì?",
    "answer": "Tự tóm tắt kiến thức, làm flashcards, giải bài tập và viết phản biện. ",
    "sim": 0.5116
  },
  {
    "rank": 2,
    "question": "sinh viên nên chuẩn bị gì trước khi đến lớp?",
    "answer": "Đọc kỹ đề cương học phần của môn học.",
    "sim": 0.335
  },
  {
    "rank": 3,
    "question": "hình phạt đối với sinh viên vắng mặt không lý do trong buổi kiểm tra quá trình là gì?",
    "answer": "Sinh viên sẽ bị nhận điểm 0 (không) cho bài kiểm tra đó.",
    "sim": 0.2538
  }
]

```

```

"query": "những gì cần chuẩn bị cho buổi học của sinh viên?",
"ground_truth": {
  "question": "sinh viên nên chuẩn bị gì trước khi đến lớp?",
  "answer": "Đọc kỹ đề cương học phần của môn học."
},
"correct_rank": 1,
"top_k": [
  {
    "rank": 1,
    "question": "sinh viên nên chuẩn bị gì trước khi đến lớp?",
    "answer": "Đọc kỹ đề cương học phần của môn học.",
    "sim": 0.8885
  },
  {
    "rank": 2,
    "question": "những việc cần làm sau khi kết thúc buổi học là gì?",
    "answer": "Tự tóm tắt kiến thức, làm flashcards, giải bài tập và viết phản biện. ",
    "sim": 0.8657
  },
  {
    "rank": 3,
    "question": "quy định về trang phục và diện mạo của sinh viên khi đến trường là gì?",
    "answer": "Trang phục phải phù hợp với quy định của nhà trường, không ăn mặc phản cảm.",
    "sim": 0.8413
  }
],

```

Hình 4.3: Log so sánh TF-IDF vs PhoBERT gốc

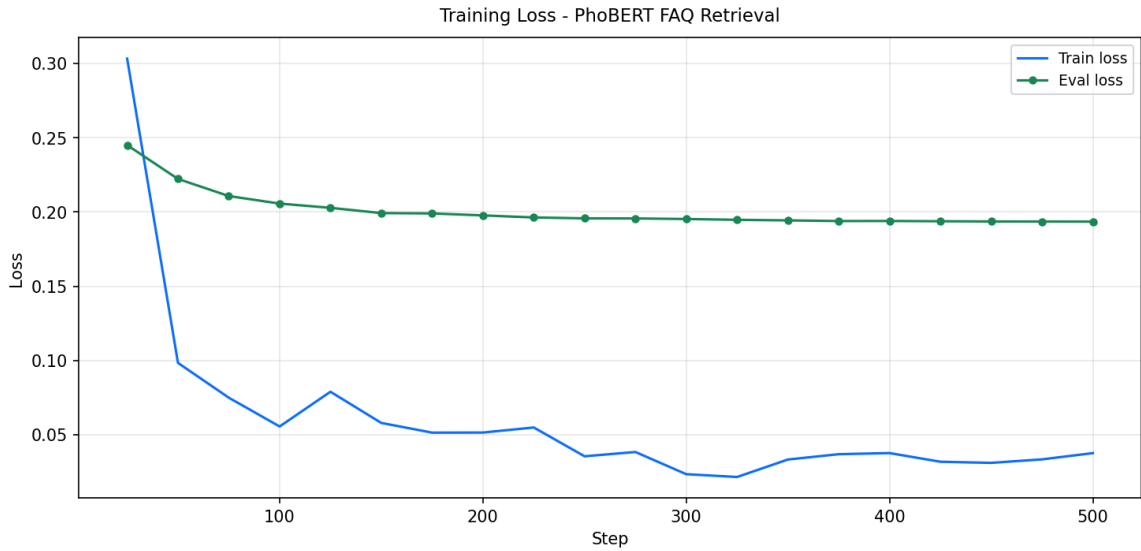
Từ đó chúng ta có thể thấy được PhoBERT gốc có khả năng hiểu ngữ nghĩa tốt hơn, nhưng sim(query, câu sai) vẫn còn cao.

PhoBERT gốc vs PhoBERT Fine-tuned: Cải thiện P@1 lên +22.70% (từ 0.7339 lên 0.9609) sau fine-tune là kết quả nổi bật nhất. Đây là minh chứng rõ ràng cho hiệu quả: chỉ với 2 epoch huấn luyện trên hàng nghìn cặp paraphrase, mô hình học được không gian embedding phù hợp với đặc thù ngôn ngữ HCMUTE FAQ.

Nghịch lý Cosine Similarity: Avg Cosine Similarity của PhoBERT fine-tuned (0.8193) thấp hơn PhoBERT gốc (0.8835) nhưng P@K lại cao hơn nhiều.

Giải thích: quá trình fine-tune với MultipleNegativesRankingLoss tái cân bằng không gian embedding – mô hình gốc có xu hướng đặt tất cả câu tiếng Việt ở vùng similarity cao (0.6–0.9), khiến Margin chỉ đạt 0.1881. Sau fine-tune, mô hình học cách phân tách mạnh hơn: sim_negative giảm sâu xuống 0.0626 (từ 0.6955), tạo Margin 0.7567 – tăng hơn 4 lần – dù sim_positive cũng giảm nhẹ.

Hiện tượng này có thể được giải thích bằng cơ chế hoạt động của MultipleNegativesRankingLoss: loss buộc mô hình phân biệt câu đúng với tất cả câu sai trong cùng batch, tạo áp lực học giảm similarity với câu sai mạnh hơn là tăng similarity với câu đúng. Kết quả là không gian embedding sau fine-tune "trải rộng" hơn, tạo điều kiện cho ranking chính xác hơn.



Hình 4.4: Đường cong loss trong quá trình huấn luyện PhoBERT Fine-tuned

Hình trên cho thấy train loss giảm mạnh từ ~0.30 xuống ~0.05 ở cuối 2 epoch (500 steps), cho thấy mô hình hội tụ tốt trên tập huấn luyện. Eval loss giảm từ ~0.245 xuống và ổn định quanh 0.195–0.200 từ khoảng step 200, không có dấu hiệu tăng trở lại – không có overfitting nghiêm trọng. Khoảng cách giữa train và eval loss phản ánh sự khác biệt phân bố giữa train/val do chia nhóm `faq_id`, không phải do overfitting.

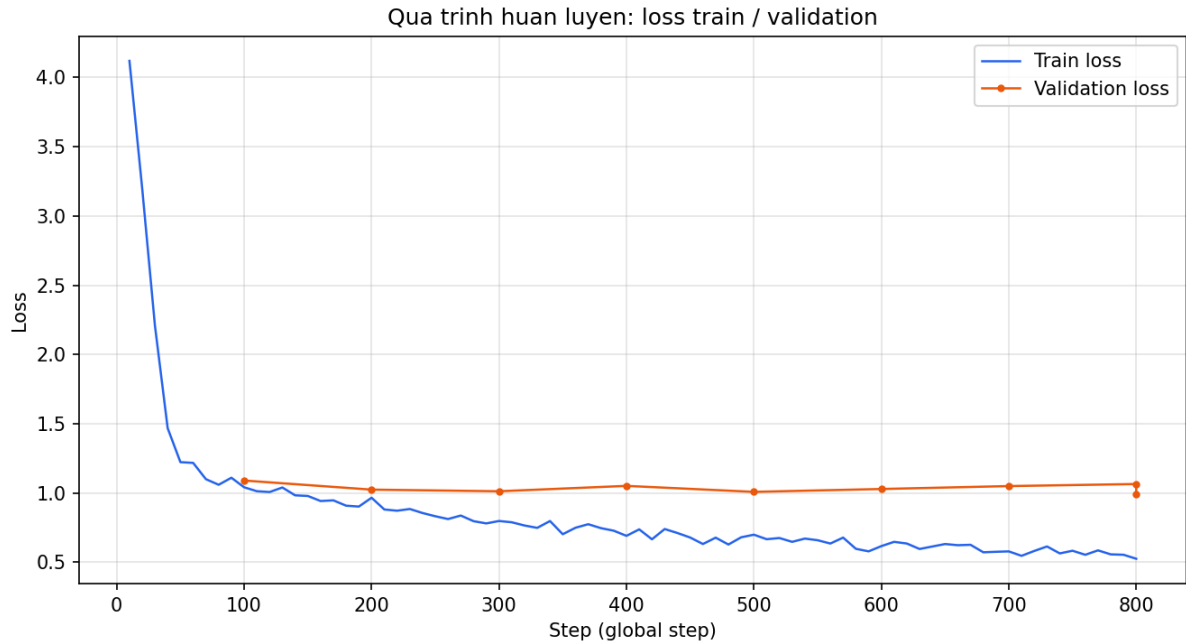
4.3.2. Phân tích kết quả fine-tune mô hình sinh

Mô hình sinh Qwen2.5-7B-Instruct được fine-tune theo phương pháp LoRA để đảm nhận vai trò generative component trong kiến trúc hybrid.

Tham số	Giá trị
Base model	unsloth/qwen2.5-7b-instruct-bnb-4bit (NF4 quantization)
LoRA rank (r)	16
LoRA alpha	16 (scaling factor = $\alpha/r = 1.0$)
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	2e-4 (warmup 3% steps + cosine decay)
Batch size (effective)	$2 \times \text{gradient_accumulation_steps} = 2$
Epochs	2
Max sequence length	2048 tokens
Optimizer	AdamW (8-bit, Unsloth-optimized)

Trainable parameters	~40M / ~7B tổng (< 0.6%)
Tổng training steps	~800 steps

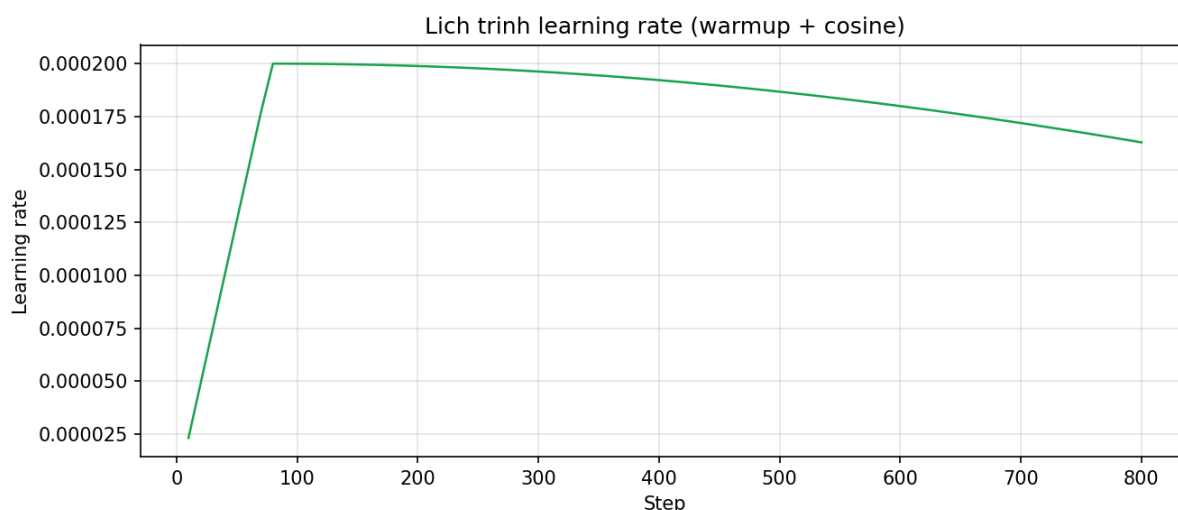
Bảng 4.3: Cấu hình *fine-tune* Qwen2.5-7B với LoRA



Hình 4.5: Đường cong loss trong quá trình *fine-tune* Qwen2.5-7B với LoRA

Hình 4.5 trình bày đường cong loss của Qwen2.5-7B trong khoảng 800 steps. Train loss giảm mạnh từ ~4.1 tại bước đầu tiên xuống ~1.0 ở khoảng step 100, phản ánh quá trình mô hình thích nghi nhanh với định dạng instruction-following của tập FAQ. Sau step 100, train loss tiếp tục giảm đều và ổn định quanh 0.5–0.6 ở cuối epoch. Validation loss dao động ổn định trong dải 1.0–1.05 sau step 100 và không tăng trong suốt quá trình huấn luyện, xác nhận mô hình không bị overfitting mặc dù tập dữ liệu tương đối nhỏ so với kích thước mô hình 7B.

Hình 4.6 cho thấy lịch trình learning rate theo chiến lược warmup + cosine decay: learning rate tăng tuyến tính từ ~2.5e-5 lên 2e-4 trong khoảng 78 steps đầu (warmup ~3% tổng steps), sau đó giảm dần theo hàm cosine xuống ~1.65e-4 ở cuối quá trình huấn luyện. Lịch trình này giúp tránh bất ổn định tối ưu (optimization instability) ở các bước đầu và đảm bảo hội tụ.



Hình 4.6: Lịch trình learning rate (warmup + cosine decay) trong quá trình fine-tune Qwen2.5-7B

Sau khi hoàn tất huấn luyện, mô hình được đánh giá trên tập test độc lập gồm 591 mẫu.

Chỉ số	Giá trị	Ý nghĩa
eval_loss (test)	0,9883	Cross-entropy trên 591 mẫu test
Corpus BLEU-4	4,885	Chất lượng khớp n-gram 4 từ liên tiếp
eval_runtime	102,6 giây	Tốc độ suy diễn
eval_samples_per_second	5,761	
Epoch tại điểm đánh giá	0,7639	

Bảng 4.4: Kết quả định lượng trên tập kiểm thử (591 mẫu)

Mô hình đạt eval_loss = 0,9883 trên tập test, sát với giá trị validation loss tốt nhất (1,011) và thấp hơn giá trị validation loss tại checkpoint cuối (1,078). Kết quả đánh giá trên test có tính đại diện và không bị ảnh hưởng bởi data leakage. Chỉ số Corpus BLEU-4 đo trên văn bản tiếng Việt đã được detokenize đạt 4,885. Đây là kết quả cần được diễn giải thận trọng trong bối cảnh bài toán sinh câu trả lời mở từ ngữ cảnh FAQ chuyên biệt và sẽ được thảo luận, phân tích một cách chi tiết, rõ ràng thông qua các đề mục bên dưới đây.

4.4. Thảo luận và Phân tích lỗi (Error Analysis)

4.4.1. Các trường hợp mô hình tốt / xấu (Lỗi thường gặp, OOV)

Mặc dù đạt kết quả tổng thể rất tốt ($P@1 = 96.09\%$), PhoBERT fine-tuned vẫn có 45 trường hợp thất bại ($\text{rank} \neq 1$), tương đương 3.91% tổng số truy vấn.

Loại lỗi	Ví dụ truy vấn	FAQ đúng bị xếp sai	Freq.
Lỗi phân biệt chủ đề gần	"học trò có bắt buộc phải làm việc thiện nguyện hay không?"	"sinh viên có phải tham gia lao động công ích không?" bị xếp dưới "tình nguyện"	~18
Paraphrase từ vựng quá xa	"tổ chức tài chính nào cung cấp hỗ trợ tài chính?"	FAQ về ngân hàng nhận học bổng không tìm thấy	~8
Thuật ngữ đặc thù / viết tắt	"khi nào có thể tổng kết số liệu cuối cùng?"	FAQ về quy trình xác nhận điểm CTXH không match	~8
Ngữ nghĩa chồng chéo	"thời gian cho vay chia thành những giai đoạn nào?"	Bị lẫn với FAQ tương tự về "thời hạn cho vay"	~11

Bảng 4.4: Phân loại các dạng lỗi chính của PhoBERT Fine-tuned

Dạng lỗi phổ biến nhất là lỗi phân biệt chủ đề ngữ nghĩa gần : hệ thống xếp sai thứ tự giữa các câu hỏi FAQ rất gần nhau về nghĩa nhưng khác nhau về khái niệm cụ thể. Ví dụ điển hình là sự nhầm lẫn giữa "lao động công ích" và "hoạt động tình nguyện" – hai khái niệm có nhiều từ khóa chung trong ngữ cảnh sinh viên nhưng là hai mục FAQ khác biệt. Khi paraphrase dùng từ như "thiện nguyện" hay "làm việc vì cộng đồng", mô hình đôi khi ưu tiên FAQ về tình nguyện hơn FAQ về lao động công ích.

Dạng lỗi thứ hai là paraphrase mạnh với từ vựng quá xa lạ (8 trường hợp): đây là các paraphrase sử dụng từ hoàn toàn khác với từ vựng xuất hiện trong FAQ, đặc biệt với các khái niệm chuyên biệt. Ví dụ, "tổ chức tài chính" thay cho "ngân hàng" trong câu hỏi về tài khoản nhận học bổng – mô hình không tìm thấy FAQ liên quan vì khoảng cách ngữ nghĩa quá lớn trong không gian embedding đã được fine-tune.

Dạng lỗi thứ ba liên quan đến thuật ngữ viết tắt và chuyên biệt (8 trường hợp): các câu hỏi sử dụng từ thông dụng ("tổng kết số liệu", "khoản tiền trợ giúp") để diễn đạt các khái niệm có tên chuyên biệt trong FAQ ("xác nhận điểm CTXH"). Sự thiếu từ vựng chuyên ngành trong tập paraphrase huấn luyện là nguyên nhân chính.

Lưu ý: Tất cả 45 trường hợp thất bại đều có FAQ đúng nằm trong top-5, không có trường hợp nào hoàn toàn bỏ sót. Điều này có nghĩa là $P@5 = 99.30\%$ – hệ thống vẫn

tìm thấy câu đúng, chỉ chưa xếp ở vị trí số 1. Trong chế độ RAG, toàn bộ top-5 được đưa vào context nên các trường hợp này vẫn được xử lý đúng ở tầng sinh câu trả lời.

4.4.2. Hạn chế

Chất lượng không đồng đều của paraphrase do LLM sinh.

Mặc dù có bước lọc là cách diễn đạt lại bằng tiếng Việt hợp lệ, một số paraphrase vẫn có chất lượng thấp: quá ngắn, không tự nhiên, hoặc thay đổi một phần ý nghĩa. Đặc biệt với các câu hỏi về số liệu cụ thể (ngày tháng, con số), LLM có xu hướng tạo ra paraphrase an toàn mà ít đa dạng về từ vựng.

Tập FAQ chưa được cập nhật liên tục (static knowledge base).

Tập FAQ HCMUTE hiện tại phản ánh thông tin tại một thời điểm cụ thể. Thông tin có thể thay đổi theo từng năm học. Hệ thống hiện tại không có cơ chế cập nhật tự động khi FAQ thay đổi.

Số lượng và chất lượng dữ liệu fine-tune.

Mô hình được huấn luyện trên ~10.000 mẫu instruction-following, một con số tương đối nhỏ so với kích thước 7B tham số. Trong điều kiện này, LoRA với chỉ 0,53% tham số cập nhật có thể chưa đủ năng lực để điều chỉnh mạnh hành vi sinh của mô hình gốc cho phù hợp với phong cách và từ vựng đặc thù của FAQ HCMUTE.

Hạn chế về khả năng bám sát nội dung FAQ.

Đánh giá định tính cho thấy mô hình có khả năng sinh câu trả lời tiếng Việt lưu loát và đúng ngữ pháp. Tuy nhiên, mức độ bám sát vào nội dung FAQ HCMUTE cụ thể chỉ đạt mức tạm chấp nhận: mô hình đôi khi sinh ra câu trả lời chung chung dựa trên kiến thức tiền huấn luyện thay vì khai thác trực tiếp thông tin từ ngữ cảnh FAQ được cung cấp.

CHƯƠNG 5: DEMO ỨNG DỤNG

5.1. Chi tiết triển khai

Hệ thống được triển khai theo kiến trúc client-server hai tầng: tầng backend xử lý toàn bộ logic truy xuất, suy diễn và sinh câu trả lời thông qua Flask REST API; tầng frontend cung cấp giao diện trò chuyện trực quan qua trình duyệt web. Hai tầng giao tiếp với nhau thông qua giao thức HTTP/JSON, đảm bảo tính tách biệt, dễ bảo trì và khả năng mở rộng độc lập.

5.1.1. Backend API bằng Flask

Backend được xây dựng bằng Flask, một micro-framework Python nhẹ và linh hoạt, phù hợp với các hệ thống AI cần tích hợp nhanh với nhiều thư viện học sâu. Toàn bộ logic xử lý được tập trung trong hai tệp chính: `app.py` và `demo.py`.

Khi ứng dụng khởi động, hàm `init_app()` được gọi để nạp mô hình PhoBERT fine-tuned từ đường dẫn cấu hình, khởi tạo kết nối ChromaDB PersistentClient và xây dựng hoặc khôi phục chỉ mục vector (collection `hcmute_faq`). Toàn bộ tài nguyên này được giữ trong bộ nhớ suốt vòng đời ứng dụng, loại bỏ overhead khởi tạo lại tại mỗi request.

API cung cấp hai endpoint chính:

POST `/ask`: Endpoint đầy đủ, trả về JSON bao gồm câu trả lời, câu hỏi chuẩn hóa, cờ `out_of_scope`, điểm khớp tốt nhất và danh sách top-K FAQ liên quan. Endpoint này được giao diện template Flask (`index.html` từ Jinja2) sử dụng.

POST `/chat`: Endpoint tương thích ngược với phiên bản frontend độc lập (`script.js`), chỉ yêu cầu trường `question` và `mode` trong JSON body.

Bốn chế độ hoạt động (mode). Hệ thống hỗ trợ bốn chế độ xử lý khác nhau, được truyền qua tham số `mode` trong request:

Mode	Tên chế độ	Mô tả logic
1	Chỉ Retrieval	Tìm kiếm vector ChromaDB, trả về câu trả lời FAQ gốc nếu <code>best_score</code> $\geq$ THRESHOLD. Nếu thấp hơn ngưỡng, trả về thông báo CANNOT_ANSWER_MSG.
2	RAG (top-k + Qwen3.5:9b)	Truy xuất top-K FAQ và đưa vào context cho LLM Qwen3.5:9b (via Ollama) sinh câu trả lời tổng hợp. Luôn gọi LLM bất kể ngưỡng.

3	Hybrid	Nếu <code>best_score ≥ THRESHOLD</code> : trả về câu trả lời FAQ trực tiếp (nhANH). Nếu <code>< THRESHOLD</code> : chuyển sang mô hình Qwen2.5-7B LoRA local sinh câu trả lời (không dùng context).
4	Chỉ mô hình sinh	Bỏ qua hoàn toàn retrieval, gọi trực tiếp <code>local_generate_answer()</code> với sampling (temperature=0.7, top_p=0.9). Phù hợp đánh giá khả năng sinh độc lập.

Bảng 5.1: Bốn chế độ xử lý câu hỏi của backend API

Server Flask được cấu hình chạy trên cổng 8080 với host 0.0.0.0, cho phép truy cập từ bên ngoài môi trường local (cần thiết khi chạy trên nền tảng đám mây như Lightning AI). Chế độ debug được tắt (`debug=False`) để tối ưu hiệu năng và bảo mật trong môi trường demo.

5.1.2. Giao diện Web (UI/UX)

Giao diện người dùng được xây dựng thuần túy bằng HTML5, CSS3 và JavaScript vanilla, không phụ thuộc vào framework frontend nào, đảm bảo tính nhẹ nhàng và tương thích rộng với các trình duyệt hiện đại. Giao diện được tích hợp vào Flask thông qua Jinja2 template engine, với tệp `index.html` được phục vụ tại route `GET /` và tệp CSS được phục vụ qua Flask's static file handler.

Hình 5.1: Giao diện chính của hệ thống

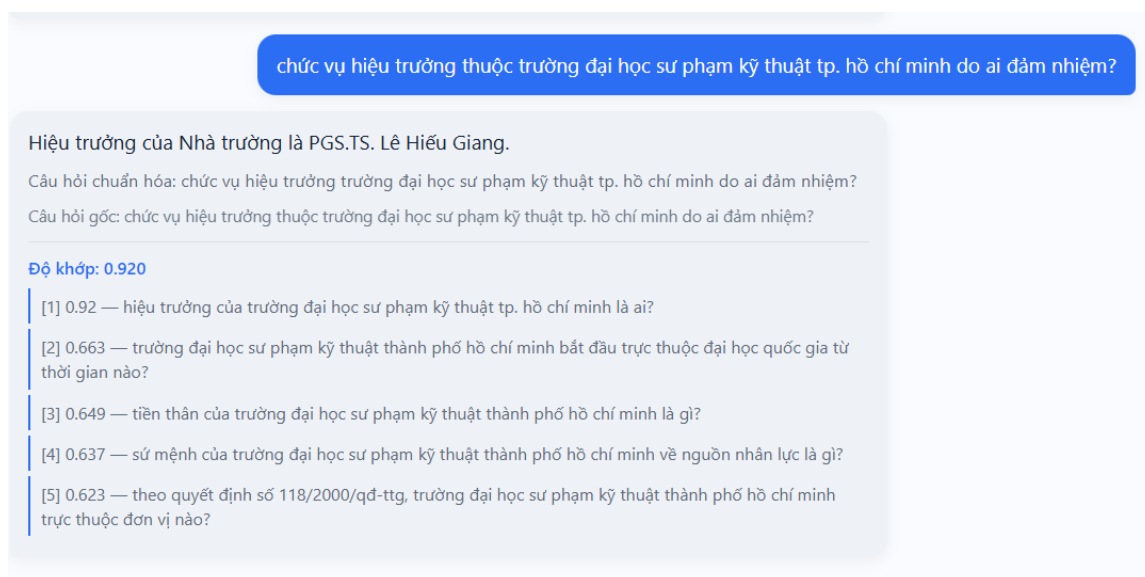
5.2. Kịch bản sử dụng thực tế

Để đánh giá hệ thống trong điều kiện thực tế, bốn kịch bản sử dụng điển hình được xây dựng dựa trên các nhóm câu hỏi phổ biến nhất trong tập. Mỗi kịch bản được kiểm thử lần lượt qua cả bốn chế độ để so sánh chất lượng phản hồi.

Kịch bản 1: Câu hỏi trong phạm vi FAQ – độ khớp cao (Mode 1)

Truy vấn: “chức vụ hiệu trưởng thuộc trường đại học sư phạm kỹ thuật tp. hồ chí minh do ai đảm nhiệm?”

Đây là kịch bản lý tưởng cho chế độ Chỉ Retrieval. Câu hỏi có từ khóa rõ ràng ("hiệu trưởng",...) và tương đồng ngữ nghĩa cao với một FAQ cụ thể trong kho tri thức. Mô hình PhoBERT fine-tuned encode câu hỏi thành embedding vector và ChromaDB trả về FAQ có cosine similarity cao. Hệ thống trả về câu trả lời gốc trực tiếp từ FAQ mà không cần qua LLM, đảm bảo tốc độ phản hồi nhanh nhất và độ chính xác thông tin cao nhất. Điểm best_score thường đạt 0.85–0.95 trong kịch bản này, giao diện hiển thị màu xanh đậm biểu thị độ tin cậy cao.



Hình 5.2: Kịch bản 1 – Mode 1 (Chỉ Retrieval), câu hỏi trong phạm vi

Kịch bản 2: Câu hỏi cần tổng hợp nhiều FAQ – Mode 2 (RAG)

Truy vấn: "Quy trình nộp đơn xin miễn giảm học phí và thời hạn nộp hồ sơ như thế nào?"

Câu hỏi này đồng thời liên quan đến quy trình thủ tục và thời hạn cụ thể, có thể phân tán thông tin trên nhiều FAQ khác nhau. Mode 2 (RAG) phù hợp nhất: hệ thống truy xuất top-5 FAQ liên quan, sau đó đưa toàn bộ context vào prompt cho Qwen3.5:9b (qua Ollama) để sinh câu trả lời tổng hợp, mạch lạc. Câu trả lời của LLM thường dài

hơn mode 1, có cấu trúc danh sách bước rõ ràng và ngôn ngữ tự nhiên hơn so với câu trả lời FAQ gốc. Đây là chế độ phù hợp khi người dùng cần giải thích chi tiết hoặc câu hỏi không thể trả lời đầy đủ chỉ từ một FAQ đơn lẻ.



Hình 5.3: Kịch bản 2 – Mode 2 (RAG), tổng hợp từ top-K FAQ thông qua Qwen3.5:9b

Kịch bản 3: Câu hỏi biến thể không có trong database – Mode 3 (Hybrid)

Truy vấn: “Tên của hiệu trưởng đương nhiệm là gì?”

Câu hỏi sử dụng cách diễn đạt khác với từ vựng trong FAQ gốc. Trong trường hợp này, `best_score` có thể nằm dưới ngưỡng `THRESHOLD` do khoảng cách từ vựng. Mode 3 (Hybrid) xử lý tình huống này hiệu quả: khi retrieval không tự tin, hệ thống tự động chuyển sang mô hình Qwen2.5-7B LoRA fine-tuned để sinh câu trả lời dựa trên kiến thức đã học từ tập FAQ HCMUTE, mà không cần context bổ sung. Cờ `out_of_scope = false` được giữ nếu mô hình sinh trả lời thành công, nhờ đó người dùng không bị thông báo lỗi không cần thiết.

Tên của hiệu trưởng đương nhiệm là gì?

Hiệu trưởng Nhà trường là Tiến sĩ Lê Hiếu Giang.

Câu hỏi chuẩn hóa: tên hiệu trưởng đương nhiệm là gì?

Câu hỏi gốc: Tên của hiệu trưởng đương nhiệm là gì?

Độ khớp: 0.559

[1] 0.559 — danh sách các phó hiệu trưởng hiện nay của trường gồm những ai?

[2] 0.54 — học hàm và học vị đầy đủ của hiệu trưởng lê hiếu giang là gì?

[3] 0.524 — học hàm và học vị của phó hiệu trưởng châu đình thành là gì?

[4] 0.515 — email chính thức của hiệu trưởng lê hiếu giang là gì?

[5] 0.503 — chức vụ đáng của phó hiệu trưởng quách thanh hải là gì?

Hybrid (dưới ngưỡng: Qwen LoRA)
Gửi

Hình 5.4: Kịch bản 3 – Mode 3 (Hybrid), fallback sang Qwen2.5-7B LoRA khi retrieval dưới ngưỡng

Kịch bản 4: Câu hỏi ngoài phạm vi – Out-of-scope

Truyện: "Thời tiết Hồ Chí Minh hôm nay thế nào?"

Đây là kịch bản câu hỏi hoàn toàn nằm ngoài domain học vụ HCMUTE. Trong Mode 1, cosine similarity giữa embedding câu hỏi và toàn bộ FAQ trong kho tri thức đều rất thấp ($\text{best_score} \ll \text{THRESHOLD}$), hệ thống trả về thông báo lịch sự (`CANNOT_ANSWER_MSG`) và gán cờ `out_of_scope = true`. Giao diện tự động nhận diện cờ này và hiển thị câu trả lời với màu đỏ cảnh báo và font chữ nghiêng, báo hiệu rõ ràng cho người dùng rằng câu hỏi nằm ngoài phạm vi hỗ trợ của hệ thống. Khả năng nhận diện out-of-scope chính xác là yêu cầu quan trọng trong hệ thống chatbot thực tế để tránh cung cấp thông tin sai lệch.

Thời tiết Hồ Chí Minh hôm nay thế nào?

Xin lỗi, tôi không thể trả lời câu hỏi này.

Câu hỏi chuẩn hóa: thời tiết tp. hồ chí minh hôm nay như thế nào?

Câu hỏi gốc: Thời tiết Hồ Chí Minh hôm nay thế nào?

Độ khớp: 0.471

[1] 0.471 — hiệu trưởng của trường đại học sư phạm kỹ thuật tp. hồ chí minh là ai?

[2] 0.457 — sinh viên đại học sư phạm kỹ thuật tp. hồ chí minh liên hệ ai khi có thắc mắc về bảo hiểm y tế?

[3] 0.415 — tầm nhìn của trường đại học sư phạm kỹ thuật thành phố hồ chí minh là gì?

[4] 0.404 — tên gọi thân thương của mái nhà chung trường đại học sư phạm kỹ thuật tp. hồ chí minh là gì?

[5] 0.4 — lối sống mà sinh viên trường đại học sư phạm kỹ thuật tp.hcm cần rèn luyện là gì?

Hình 5.5: Kịch bản 4 – Mode 1

Thời tiết Hồ Chí Minh hôm nay thế nào?

Thông tin về thời tiết tại TP. Hồ Chí Minh hôm nay không có trong [THÔNG TIN FAQ] cung cấp, nên tôi không thể trả lời câu hỏi này dựa trên dữ liệu đã cho.

Câu hỏi chuẩn hóa: thời tiết tp. hồ chí minh hôm nay như thế nào?

Câu hỏi gốc: Thời tiết Hồ Chí Minh hôm nay thế nào?

Độ khớp: 0.471

- [1] 0.471 — hiệu trưởng của trường đại học sư phạm kỹ thuật tp. hồ chí minh là ai?
- [2] 0.457 — sinh viên đại học sư phạm kỹ thuật tp. hồ chí minh liên hệ ai khi có thắc mắc về bảo hiểm y tế?
- [3] 0.415 — tầm nhìn của trường đại học sư phạm kỹ thuật thành phố hồ chí minh là gì?
- [4] 0.404 — tên gọi thân thương của mái nhà chung trường đại học sư phạm kỹ thuật tp. hồ chí minh là gì?
- [5] 0.4 — lối sống mà sinh viên trường đại học sư phạm kỹ thuật tp.hcm cần rèn luyện là gì?

Hình 5.6: Kịch bản 4 – Mode 2

Thời tiết Hồ Chí Minh hôm nay thế nào?

Dự báo thời tiết: Mây rải đều, có nơi có giông. Nhiệt độ dao động từ 27 đến 31 độ Celsius. Gió đông nam cấp 2.

Câu hỏi chuẩn hóa: thời tiết tp. hồ chí minh hôm nay như thế nào?

Câu hỏi gốc: Thời tiết Hồ Chí Minh hôm nay thế nào?

Độ khớp: 0.471

- [1] 0.471 — hiệu trưởng của trường đại học sư phạm kỹ thuật tp. hồ chí minh là ai?
- [2] 0.457 — sinh viên đại học sư phạm kỹ thuật tp. hồ chí minh liên hệ ai khi có thắc mắc về bảo hiểm y tế?
- [3] 0.415 — tầm nhìn của trường đại học sư phạm kỹ thuật thành phố hồ chí minh là gì?
- [4] 0.404 — tên gọi thân thương của mái nhà chung trường đại học sư phạm kỹ thuật tp. hồ chí minh là gì?
- [5] 0.4 — lối sống mà sinh viên trường đại học sư phạm kỹ thuật tp.hcm cần rèn luyện là gì?

Hình 5.7: Kịch bản 4 – Mode 3

PHẦN KẾT LUẬN

1. Đánh giá kết quả

Hệ thống hỏi đáp FAQ dành cho sinh viên HCMUTE được xây dựng trên nền tảng kiến trúc kết hợp giữa mô hình truy xuất ngữ nghĩa và mô hình ngôn ngữ sinh, cho thấy hiệu quả rõ rệt so với các phương pháp truyền thống trong việc xử lý ngôn ngữ tiếng Việt.

1.1. Ưu điểm

Hiệu quả truy xuất thông tin được cải thiện rõ rệt nhờ quá trình fine-tune PhoBERT. Với chỉ số $P@1$ đạt 96,09% và $P@5$ lên tới 99,30% trên tập kiểm thử độc lập gồm 1.150 truy vấn, mô hình chứng minh khả năng ánh xạ ngữ nghĩa cực kỳ chính xác giữa câu hỏi của sinh viên và tập câu hỏi chuẩn trong database. Đây là kết quả đáng ghi nhận trong bối cảnh tiếng Việt vốn có cấu trúc ngôn ngữ đơn lập và nhiều biến thể diễn đạt cho cùng một ý nghĩa.

Khả năng hiểu ngữ nghĩa (semantic similarity) của hệ thống thể hiện rõ ràng thông qua cơ chế biểu diễn vector trong không gian nhúng dày (dense embedding), kết hợp với cơ sở dữ liệu vector ChromaDB. Hệ thống không chỉ khớp từ khóa bề mặt mà còn nắm bắt được ý định và nghĩa hàm ẩn phía sau các cách diễn đạt khác nhau của sinh viên - một yêu cầu thiết yếu đối với hệ thống hỏi đáp tự động trong thực tế. Điều này được kiểm chứng qua việc tất cả 45 trường hợp lỗi của PhoBERT đều có câu trả lời đúng nằm trong top-5, đồng nghĩa với việc hệ thống không bỏ sót thông tin liên quan ngay cả trong các tình huống khó nhất.

Kết quả fine-tune trên cả hai thành phần bao gồm mô hình retrieval lẫn mô hình sinh đều cho thấy chiến lược huấn luyện được thiết kế hợp lý và hiệu quả. Mô hình PhoBERT fine-tuned với contrastive learning trên tập paraphrase ~10.000 mẫu đã hội tụ tốt, không có dấu hiệu overfitting nghiêm trọng khi eval loss ổn định quanh 0,195–0,200. Tương tự, mô hình Qwen2.5-7B được tinh chỉnh hiệu quả tham số bằng LoRA (chỉ 0,53% tham số được cập nhật) nhưng vẫn đạt validation loss ổn định trong dải 1,0–1,05 suốt toàn bộ quá trình huấn luyện.

Về tính ứng dụng thực tiễn, hệ thống được triển khai theo kiến trúc client-server hai tầng với Flask REST API, hỗ trợ bốn chế độ hoạt động linh hoạt - từ truy xuất thuần túy (Mode 1) cho đến RAG đầy đủ (Mode 2) và mô hình lai thích nghi (Mode 3) - cho phép cân bằng tốt giữa tốc độ phản hồi và chất lượng câu trả lời tùy theo tình huống cụ

thể. Hệ thống cũng tích hợp cơ chế nhận diện câu hỏi ngoài phạm vi một cách tự động và trực quan, đảm bảo tính tin cậy trong triển khai thực tế.

1.2. Nhược điểm

Bên cạnh những kết quả đáng khích lệ, hệ thống vẫn tồn tại một số hạn chế khách quan vốn phổ biến trong các nghiên cứu NLP ứng dụng hiện nay. Chất lượng của tập dữ liệu paraphrase được sinh bởi LLM chưa hoàn toàn đồng đều bởi vì trong đó còn tồn tại một số mẫu còn ngắn, ít đa dạng về từ vựng, đặc biệt với các câu hỏi liên quan đến số liệu hoặc thuật ngữ chuyên biệt. Đây là thách thức mang tính hệ thống trong các pipeline sinh dữ liệu tự động và cũng chính là khoảng trống mà các phương pháp lọc và kiểm định dữ liệu tiên tiến hơn có thể bổ sung trong tương lai. Ngoài ra, mô hình sinh Qwen2.5-7B sau fine-tune đôi khi vẫn có xu hướng sinh câu trả lời dựa trên kiến thức tiền huấn luyện thay vì bám sát nội dung FAQ được cung cấp trong ngữ cảnh - một hạn chế phổ biến của các mô hình ngôn ngữ lớn khi miền dữ liệu fine-tune còn tương đối hẹp.

2. Hướng phát triển

Xuất phát từ những hạn chế được nhận diện ở trên, nhóm xác định một số hướng nâng cấp có giá trị cao và tính khả thi lớn trong tương lai gần. Trước tiên, việc mở rộng và đa dạng hóa tập dữ liệu FAQ bao gồm bổ sung câu hỏi từ nhiều khóa học và năm học khác nhau, tích hợp cơ chế cập nhật tri thức tự động khi chính sách nhà trường thay đổi sẽ trực tiếp cải thiện độ bao phủ và tính thời sự của hệ thống. Song song đó, việc xây dựng pipeline kiểm soát chất lượng dữ liệu paraphrase tự động dựa trên các chỉ số ngữ nghĩa (semantic similarity threshold, lexical diversity) sẽ nâng cao đáng kể chất lượng tập huấn luyện mà không cần can thiệp thủ công.

Về khía cạnh kỹ thuật, pipeline RAG có thể được cải tiến theo nhiều hướng: áp dụng kỹ thuật re-ranking để tối ưu hóa thứ tự trả về của ChromaDB, sử dụng chiến lược context compression để chỉ đưa vào prompt những đoạn thông tin thực sự liên quan, hoặc tích hợp các cơ chế kiểm soát hallucination để buộc mô hình sinh bám sát nội dung FAQ được cung cấp. Ngoài ra, việc thử nghiệm với các mô hình nền mạnh hơn (chẳng hạn Qwen2.5-14B hoặc các mô hình tiếng Việt chuyên biệt) cùng kỹ thuật fine-tune đầy đủ tham số trên cơ sở hạ tầng đám mây sẽ mở ra không gian cải thiện đáng kể về chất lượng sinh câu trả lời.

TÀI LIỆU THAM KHẢO

- [1] D. Q. Nguyen and A. Nguyen, “PhoBERT: Pre-trained Language Models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online, 2020, pp. 1037–1042.

BẢNG KHỐI LƯỢNG CÔNG VIỆC

Họ và tên	Công việc chủ yếu
Huỳnh Thanh Nhân	Thảo luận tìm hướng làm đề án, thu thập câu hỏi từ dữ liệu, sinh dữ liệu bằng LLM(paraphrase thông thường), tiền xử lý, huấn luyện PhoBERT, đánh giá PhoBERT fine-tune và mô hình sinh, viết báo cáo phản tương ứng.
Trương Tấn Sang	Thảo luận tìm hướng làm đề án, thu thập câu hỏi từ dữ liệu, sinh dữ liệu bằng LLM (paraphrase mạnh), tiền xử lý, code và đánh giá TF-IDF, xây dựng giao diện demo, viết báo cáo phản tương ứng.
Huỳnh Ngọc Thắng	Thảo luận tìm hướng làm đề án, thu thập câu hỏi từ dữ liệu, tiền xử lý, huấn luyện mô hình sinh, đánh giá mô hình sinh và PhoBERT gốc, viết báo cáo phản tương ứng.